

TÀI LIỆU KIẾN TRÚC KỸ THUẬT

TalentChart

Phase 1-2 Technical Architecture

Build mới với AI tạo sinh, kế thừa tinh túy từ

SmartHire + Fortune HR + EPA Engine

Tài liệu này dành cho:

- Mid Developer chính (sẽ implement)
- 2-3 sinh viên CNTT HPU (intern)
- Trung — Tech Lead / Architect / DevOps / PM

Đọc trước khi viết bất kỳ dòng code nào

Phiên bản 2.0 · Tháng 4 năm 2026 · HPU & SCEI

v2.0: Đối sang chiến lược BUILD MỚI (không migrate)

MỤC LỤC

Tóm tắt kiến trúc cho người đọc nhanh.....	3
1. Trích xuất giá trị từ codebase hiện tại.....	5
1.1. Triết lý: Reuse knowledge, không reuse code.....	5
1.2. SmartHire — Tinh túy cần kế thừa.....	6
1.3. Fortune HR — Tinh túy cần kế thừa.....	8
1.4. Pipeline 9 → 7 trạng thái.....	10
2. Nguyên tắc thiết kế kiến trúc.....	11
3. Tổng quan kiến trúc TalentChart Phase 1.....	13
4. Multi-Tenant — Trái tim của Phase 1.....	17
5. Database Schema chi tiết.....	23
6. EPA Engine — USP cốt lõi.....	28
7. API Design.....	33
8. Security & Compliance.....	36
9. Async, Queue, Kafka — khi nào dùng cái nào.....	38
10. Data import từ DB cũ (không phải migration).....	41
11. Deployment với caddy-docker-proxy.....	44

TÓM TẮT KIẾN TRÚC CHO NGƯỜI ĐỌC NHANH

Nếu chỉ có 5 phút, đọc phần này:

Quyết định kiến trúc cốt lõi (v2.0 — đã cập nhật)

- BUILD MỚI HOÀN TOÀN bằng AI tạo sinh (Claude Sonnet 4.7) — KHÔNG migrate code từ SmartHire / Fortune HR. Lý do: vibe coding với architect cẩn thận có thể sinh code clean trong 1-2 tuần cho cùng phạm vi mà migrate + refactor mất 2-3 tuần và để lại technical debt.
- KẾ THỪA tinh túy từ codebase cũ dưới dạng kiến thức (knowledge): thuật toán DISC scoring, engine LunarData 200 năm, logic Tam hợp / Xung, bộ câu hỏi 40 DISC + 30 personality, ngôn ngữ Tiếng Việt cho UI / email.
- KẾ THỪA dữ liệu từ DB cũ qua export JSON / CSV → import vào TalentChart sau khi build xong. Không cần migration script phức tạp, không cần parallel run.
- Stack: FastAPI + PostgreSQL 16 + Redis 7 + Celery cho background jobs. Frontend Next.js 15 + TypeScript + Tailwind + shadcn / ui.
- Multi-tenant với shared schema + Row Level Security là yêu cầu quan trọng nhất Phase 1.
- Pipeline gọn lại 7 trạng thái thay vì 9 (gộp 3 round interview thành 1 với sub-round number).
- Deploy Docker Compose với caddy-docker-proxy (lucaslorentz) — pattern label-based mà Trung đã có sẵn cho 12 server.
- KHÔNG dùng Kafka Phase 1-2 — Redis + Celery đủ. Kafka chỉ cần nhắc khi >10K event / giây hoặc Phase 3+ scale lên 500+ customer.

3 nguy cơ kỹ thuật lớn nhất Phase 1

Nguy cơ 1 — Multi-tenant data leak

Một query thiếu filter organization_id có thể làm dữ liệu trường A lọt sang trường B. Đây là bug duy nhất có thể giết startup ngay lập tức.

Mitigation:

Dùng SQLAlchemy event listener tự inject WHERE organization_id = current_org_id() vào MỌI query + PostgreSQL Row Level Security làm lớp 2. Test isolation bắt buộc trong CI.

Nguy cơ 2 — Vibe coding sinh ra security holes

Junior + AI có thể sinh ra code chạy được nhưng có SQL injection, XSS, IDOR (Insecure Direct Object Reference). Đặc biệt đáng lo khi build mới hoàn toàn — không có bộ khung security đã được kiểm chứng làm 'rào chắn'.

Mitigation:

Mỗi PR phải qua security checklist (chi tiết ở Vibe Coding Playbook). Trung là người duy nhất merge vào main. Bắt buộc dùng SQLAlchemy ORM, không dùng raw SQL. Có security tests chạy tự động trong CI.

Nguy cơ 3 — EPA Engine sai thuật toán bất tự

LunarData 200 năm và logic Can Chi rất tinh tế. Một bug nhỏ (ví dụ dùng SOLAR YEAR thay vì LUNAR YEAR cho người sinh trước Tết) có thể làm 50% kết quả EPA sai mà không phát hiện ra ngay.

Mitigation:

Port LunarData NGUYỄN XI từ Fortune HR (kế thừa kiến thức), viết test case so sánh output Python với output JS gốc cho 1000 ngày sinh ngẫu nhiên 1900-2100. Phải pass 100% trước khi đưa vào production.

Phân bổ effort 4 tháng Phase 1 (v2.0)

Hoạt động	% effort	Lý do
Build core (auth, multi-tenant, candidates, campaigns)	30%	Backbone của toàn bộ hệ thống
Build Module 1 mới (Job Board + Career Page)	20%	Module hoàn toàn mới
EPA Engine port + 12 archetype fusion logic	20%	USP cốt lõi
Frontend Next.js 15 (admin + public pages)	15%	UI hiện đại thay HTML cũ
Data import từ DB cũ + testing + security hardening	15%	Đảm bảo HPU dùng được ngay

Tổng effort ước tính: ~80 ngày người (1 mid dev fulltime + 2 intern part-time × 4 tháng) hoàn toàn khả thi với vibe coding.

1. TRÍCH XUẤT GIÁ TRỊ TỪ CODEBASE HIỆN TẠI

1.1. Triết lý: Reuse knowledge, không reuse code

Quyết định kiến trúc số 1 của TalentChart Phase 1

BUILD MỚI hoàn toàn bằng AI tạo sinh (Claude Sonnet 4.7), KHÔNG migrate code từ SmartHire / Fortune HR. Tuy nhiên, KẾ THỪA tối đa các tài sản phi-code: thuật toán, dữ liệu, kiến thức nghiệp vụ, nội dung Tiếng Việt.

Tại sao build mới nhanh hơn migrate?

Một, vibe coding đã thay đổi tỷ lệ effort. Trước AI tạo sinh, viết code mới là phần tốn 70% effort. Sau Claude Sonnet 4.7, viết code mới chỉ tốn 20% effort — phần lớn thời gian dev là review và sửa. Trong khi đó, migrate vẫn cần đọc hiểu code cũ + viết script chuyển + test compatibility — không có AI giúp được nhiều ở phần đầu.

Hai, code cũ có technical debt. SmartHire và Fortune HR được build cho HPU single-tenant. Mọi query, route, schema đều giả định 1 tổ chức duy nhất. Refactor multi-tenant là sửa khắp nơi, dễ sót. Build mới với multi-tenant từ đầu thì pattern đồng nhất, không có chỗ để sót.

Ba, build mới cho phép thiết kế kiến trúc tốt hơn ngay. Khi migrate, ta bị ép theo cấu trúc cũ. Khi build mới, ta thiết kế cho mục đích thực sự (SaaS multi-tenant, scale 200 customer). Code clean architecture từ đầu = ít technical debt = ít chi phí maintain dài hạn.

Bốn, team đang xây năng lực qua vibe coding. Việc 1 mid dev + 2-3 intern thực sự build từ đầu với Claude tạo ra learning value cao. Migrate chỉ là việc sửa kỹ thuật, không tạo ra hiểu biết sâu về sản phẩm.

Cái gì kế thừa, cái gì bỏ?

Loại tài sản	SmartHire	Fortune HR	Cách kế thừa
Code Python / JS	✗ Bỏ	✗ Bỏ	Build lại bằng vibe coding
Database schema design	✓ Tham khảo	✓ Tham khảo	Đọc model, viết lại tốt hơn
DISC scoring algorithm	✓ Kế thừa	—	Port nguyên xi qua Python
LunarData 200 năm	—	✓ Kế thừa	Port nguyên xi qua Python
Logic Tam hợp / Xung	—	✓ Kế thừa	Port nguyên xi qua Python
Logic Nạp Âm / Mệnh	—	✓ Kế thừa	Port nguyên xi qua Python
40 câu hỏi DISC	✓ Kế thừa	—	Copy nội dung Tiếng Việt
30 câu hỏi Personality	✓ Kế thừa	—	Copy nội dung Tiếng Việt

Loại tài sản	SmartHire	Fortune HR	Cách kế thừa
12 trang HTML SmartHire	✗ Bỏ	—	Build lại Next.js
JD templates / email VN	✓ Kế thừa	—	Copy nội dung
Pipeline state model	⚠ Cải thiện	—	Giảm 9 → 7 trạng thái
Dữ liệu users / candidates	✓ Import	✓ Import	Export JSON → import sau
Dữ liệu CB HPU + ngày sinh	—	✓ Import	Export → import → tính EPA tự động

1.2. SmartHire — Tinh túy cần kế thừa

SmartHire là một sản phẩm chất lượng cao đang chạy production tại hire.hpu.edu.vn. Mặc dù không reuse code, có rất nhiều giá trị có thể kế thừa:

Tài sản 1 — Stack design đã được kiểm chứng

SmartHire chứng minh rằng FastAPI + PostgreSQL + Redis + SQLAlchemy async là combo phù hợp cho HR Tech tại VN. Đây là input cho quyết định stack của TalentChart (giữ nguyên stack).

Tài sản 2 — DISC scoring algorithm

Đoạn code calculate_disc() trong services / disc.py là một thuật toán scoring chuẩn, đã test trên thực tế với hàng trăm ứng viên HPU. Cụ thể:

- Most / least selection: ứng viên chọn 1 trong 4 option là 'most like me' và 1 là 'least like me' cho mỗi câu (40 câu)
- Scoring: most = +1 điểm cho dimension đó, least = -0.5 điểm
- Normalization: shift về 0, sum = 100%, ra phần trăm 4 chiều D / I / S / C
- Profile: primary là cao nhất, secondary nếu cao bằng 70% của primary

TalentChart sẽ implement lại logic này trong Python với type hints và unit tests đầy đủ, không reuse code SmartHire trực tiếp.

Tài sản 3 — Bộ câu hỏi 40 DISC + 30 Personality

File data / questions.py chứa 40 câu DISC + 30 câu Personality bằng Tiếng Việt — đây là content đã được Trung và team HPU validate qua nhiều đợt tuyển dụng. Tốn rất nhiều effort để tạo và validate. TalentChart copy nguyên văn nội dung Tiếng Việt + cấu trúc data structure.

Tài sản 4 — Pipeline state model (cần cải thiện)

SmartHire hiện có 9 trạng thái. Quá chi tiết, đặc biệt 3 trạng thái interview làm UI rườm rà. TalentChart áp dụng pipeline 7 trạng thái (chi tiết ở mục 1.4 dưới đây).

Tài sản 5 — RBAC concept

Roles của SmartHire: super_admin, hr_manager, hr_staff, bgh, interviewer. TalentChart kế thừa concept và thiết kế lại 2 lớp: System role (global) và Organization role (theo tổ chức) — chi tiết ở chương 4.

Tài sản 6 — JD templates và email content Tiếng Việt

SmartHire có sẵn templates email (gửi link test, mời phỏng vấn, kết quả) và JD samples đã viết bằng Tiếng Việt chuẩn HR. Copy nguyên văn.

1.3. Fortune HR — Tinh túy cần kế thừa

Fortune HR chứa kim cương — engine bất tự không dễ viết lại

Toàn bộ logic lịch âm, Can Chi, Nạp Âm, Tam hợp đã được kiểm chứng production. Viết lại từ đầu sẽ tốn 2-3 tuần và rất dễ sai. Ta KẾ THỪA logic này nguyên xi qua Python.

Tài sản 1 — LunarData 200 năm (1900-2100)

Mảng hex-encoded chứa thông tin tháng nhuận và số ngày mỗi tháng âm lịch cho 200 năm. Đây là knowledge chuẩn của các engine lịch âm Việt Nam, đã được kiểm chứng ở nhiều phần mềm.

- Mỗi entry là 1 số 32-bit encode: tháng nhuận, số ngày mỗi tháng (29 hoặc 30), số ngày tháng nhuận
- Bit operation để decode: getLeapMonth(y), getLunarYearDays(y), getLunarMonthDays(y, m)
- Port nguyên xi sang Python (file lunar.py), không sửa

Tài sản 2 — Bug fix Lunar Year cho người sinh trước Tết

Bug fix critical:

Nếu một người sinh ngày 1/1/1938 dương lịch, nhiều phần mềm tính họ là tuổi Mậu Dần (1938 âm). NHƯNG thực tế họ là tuổi Đinh Sửu (1937 âm) — vì Tết 1938 rơi vào 31/1/1938 dương, người sinh trước đó vẫn thuộc năm âm 1937.

Code Fortune HR đã fix bằng cách dùng LUNAR YEAR (đã convert) thay vì SOLAR YEAR khi tính Can Chi. TalentChart KẾ THỪA fix này.

Tài sản 3 — Logic 60 hoa giáp (Nạp Âm) → 5 mệnh ngũ hành

Mảng NAP_AM_60 (Hải Trung Kim, Lư Trung Hỏa...) và mapping MENH_BY_NAP_AM (Kim / Mộc / Thủy / Hỏa / Thổ → set các nạp âm thuộc về mệnh đó). Port nguyên xi.

Tài sản 4 — Logic Tam hợp / Xung

- 4 nhóm Tam hợp: Thân-Tý-Thìn, Ty-Dậu-Sửu, Dần-Ngo-Tuất, Hợi-Mão-Mùi

- 6 cặp Xung: Tý-Ngọ, Sửu-Mùi, Dần-Thân, Mão-Dậu, Thìn-Tuất, Ty-Hợi
- Thuật toán scoring tương hợp 2 người: 50 điểm base + 25 nếu Tam hợp - 30 nếu Xung, clamp 0-100
- Algorithm team-suggest tạo và đánh giá nhiều combination để tìm team tương hợp tốt nhất

Tài sản 5 — Dữ liệu CB HPU 150+ người với ngày-tháng-năm sinh

DB Fortune HR có dữ liệu thực 150+ cán bộ HPU (đã loại bỏ entries lỗi như 'Vũ Văn Hải'). Đây là dataset thực để TalentChart test EPA Engine và có data ngay khi go-live, không cần seed manual.

1.4. Pipeline 9 → 7 trạng thái

Cải thiện so với SmartHire

Pipeline cũ 9 trạng thái dài và rườm. Pipeline mới 7 trạng thái gọn, dùng sub-round number cho các vòng phỏng vấn nếu cần.

Pipeline cũ (SmartHire — 9 trạng thái)

```
NEW
→ TEST_SENT
  → TEST_COMPLETED
    → WAITING_INTERVIEW
      → INTERVIEW_ROUND_1
        → INTERVIEW_ROUND_2
          → DECISION
            → HIRED / REJECTED
```

Pipeline mới (TalentChart — 7 trạng thái)

```
NEW          -- Vừa tiếp nhận hồ sơ
→ SCREENING   -- Đang sàng lọc / chờ test
  → TEST_SENT -- Đã gửi link test (gộp WAITING)
    → TEST_DONE -- Đã làm test (đã có DISC + EPA)
      → INTERVIEW -- Đang phỏng vấn (1+ vòng, dùng interview_round)
        → DECISION -- Hội đồng quyết định
          → HIRED / REJECTED
```

Vòng phỏng vấn xử lý bằng sub-data thay vì status

Một số vị trí có 1 vòng phỏng vấn, một số có 2-3 vòng. Thay vì để ra INTERVIEW_ROUND_1, INTERVIEW_ROUND_2, INTERVIEW_ROUND_3, ta dùng:

- Status: INTERVIEW (đơn giản)
- Field interview_round: integer (1, 2, 3...) trong bảng interview_scores

- UI hiển thị: tab 'Vòng 1', 'Vòng 2'... bên trong status INTERVIEW
- Khi cần báo cáo: query interview_scores với GROUP BY interview_round

Cách này linh hoạt hơn — vị trí 1 vòng không cần phải skip status, vị trí 4 vòng không cần thêm status mới.

Mapping data từ SmartHire cũ → TalentChart

SmartHire (cũ)	TalentChart (mới)	Logic
NEW	NEW	Giữ nguyên
TEST_SENT	TEST_SENT	Giữ nguyên
TEST_COMPLETED	TEST_DONE	Đổi tên cho ngắn
WAITING_INTERVIEW	TEST_DONE	Gộp — không cần riêng
INTERVIEW_ROUND_1	INTERVIEW + round=1	Gộp
INTERVIEW_ROUND_2	INTERVIEW + round=2	Gộp
DECISION	DECISION	Giữ nguyên
HIRED	HIRED	Giữ nguyên
REJECTED	REJECTED	Giữ nguyên

2. NGUYÊN TẮC THIẾT KẾ KIẾN TRÚC

Mọi quyết định kiến trúc trong dự án phải tuân theo 8 nguyên tắc dưới đây. Khi gặp ngã rẽ kỹ thuật, dùng các nguyên tắc này để quyết định, không cãi nhau theo sở thích cá nhân.

Nguyên tắc 1 — Multi-tenant safety là ưu tiên cao nhất

Không có nguyên tắc nào được phép đứng trên multi-tenant safety. Một bug data leak giữa các trường có thể giết startup ngay lập tức và gây ra hậu quả pháp lý theo Nghị định 13/2023/NĐ-CP. Cụ thể:

- Mọi bảng dữ liệu khách hàng đều phải có cột `organization_id` NOT NULL với foreign key về bảng `organizations`.
- Mọi query `SELECT/UPDATE/DELETE` phải có filter `organization_id = current_org_id()` — enforce ở SQLAlchemy event listener, không tin code application.
- Mọi API endpoint nhận id của resource (ví dụ `candidate_id`) phải verify resource đó thuộc về org hiện tại trước khi trả về dữ liệu.
- Code review tự động phải fail nếu có raw SQL không có org filter.
- Test multi-tenancy là test bắt buộc trong CI: tạo 2 org, kiểm tra data của org A không được trả về cho user của org B.

Nguyên tắc 2 — Reuse knowledge, không reuse code

Build mới hoàn toàn bằng vibe coding, KHÔNG migrate code từ codebase cũ. Tuy nhiên, kế thừa tối đa các tài sản phi-code:

- Thuật toán đã được kiểm chứng (DISC scoring, Lunar conversion, Tam hợp logic) — port LOGIC nguyên xi qua Python, không sửa logic vì đã chạy production ổn định.
- Nội dung Tiếng Việt (40 câu DISC, 30 câu personality, JD templates, email templates) — copy nguyên văn.
- Dữ liệu thực (users HPU, candidates, cán bộ) — export JSON / CSV và import vào TalentChart sau khi build xong.
- Database schema design — tham khảo, không copy. Build lại với multi-tenant từ đầu.
- Kinh nghiệm UX (bug đã fix ở Fortune HR v6.2) — áp dụng ngay vào thiết kế Frontend Next.js.

Mỗi lần dev định viết một thuật toán mà SmartHire / Fortune HR đã có, hỏi câu này trước: thuật toán cũ đã được kiểm chứng production chưa? Nếu rồi, port nguyên xi knowledge — không sáng tạo ở Phase 1.

Nguyên tắc 3 — Async-first cho mọi I/O

Stack đã chọn FastAPI + SQLAlchemy 2.0 async + asyncpg. Mọi database query, HTTP call, file I/O đều phải dùng `async/await`. Cụ thể:

- Database session phải là `AsyncSession` (đã có sẵn trong SmartHire `database.py`).
- HTTP call sang Anthropic API dùng `httpx.AsyncClient` (đã có `httpx` trong requirements).
- Email gửi dùng `aiosmtplib` (đã có).

- File upload streaming bằng async, không load toàn bộ file vào RAM.
- Background task nặng (parse CV, sinh PDF) dùng FastAPI BackgroundTasks hoặc Celery (Phase 2).

Nguyên tắc 4 — Vibe-coding-friendly architecture

Kiến trúc phải được thiết kế để dev junior + Cursor có thể implement chính xác. Cụ thể:

- Mỗi module có cấu trúc folder giống nhau (router/, service/, schemas/, models/) — junior chỉ cần copy pattern.
- Mọi router endpoint dài tối đa 30 dòng, business logic nặng đẩy vào service layer.
- Naming convention nhất quán: snake_case Python, kebab-case URL, PascalCase Pydantic schemas.
- Mọi function có docstring + type hints — Cursor dùng những thông tin này để gợi ý chính xác.
- Mỗi feature có ít nhất 1 example file (router + service + test) trong code base làm reference cho dev hỏi Cursor 'làm giống như cái này'.

Nguyên tắc 5 — Fail-fast và explicit errors

- Config phải fail ngay khi startup nếu thiếu env quan trọng (DATABASE_URL, JWT_SECRET, ANTHROPIC_API_KEY).
- Không dùng default fallback cho secrets như SmartHire hiện tại.
- Pydantic validation cho mọi request input — không trust client.
- Custom exceptions có HTTP status code rõ ràng (TenantAccessDenied → 403, ResourceNotFound → 404, ...).
- Không catch generic Exception và swallow lỗi — luôn log đầy đủ.

Nguyên tắc 6 — Database migration nghiêm túc

- Mọi schema change phải qua Alembic migration, không sửa DB trực tiếp.
- Mỗi migration phải có cả upgrade() và downgrade() function.
- Test migration trên copy của production data trước khi chạy thật.
- Backup DB tự động hằng ngày + giữ 7 ngày gần nhất + test restore định kỳ.

Nguyên tắc 7 — Observability ngay từ đầu

- Structured logging với context (tenant_id, user_id, request_id) — dùng structlog hoặc loguru.
- Mọi API call ghi log: method, path, status, duration, tenant_id.
- Mọi background task ghi log: start, end, duration, error nếu có.
- Health check endpoint /api/v1/health trả về trạng thái DB, Redis, external APIs.
- Sentry hoặc tương đương cho error tracking — Phase 1 có thể dùng self-hosted Sentry để tiết kiệm.

Nguyên tắc 8 — Docker-first development

- Mọi service chạy trong Docker, kể cả khi dev local.
- `docker-compose.dev.yml` cho development (mount source code, hot reload), `docker-compose.prod.yml` cho production.
- Một câu lệnh `docker compose up -d` phải dựng được toàn bộ stack từ con số 0.
- Caddy label-based routing cho production (đã có pattern từ Fortune HR).

3. TỔNG QUAN KIẾN TRÚC TALENTCHART PHASE 1

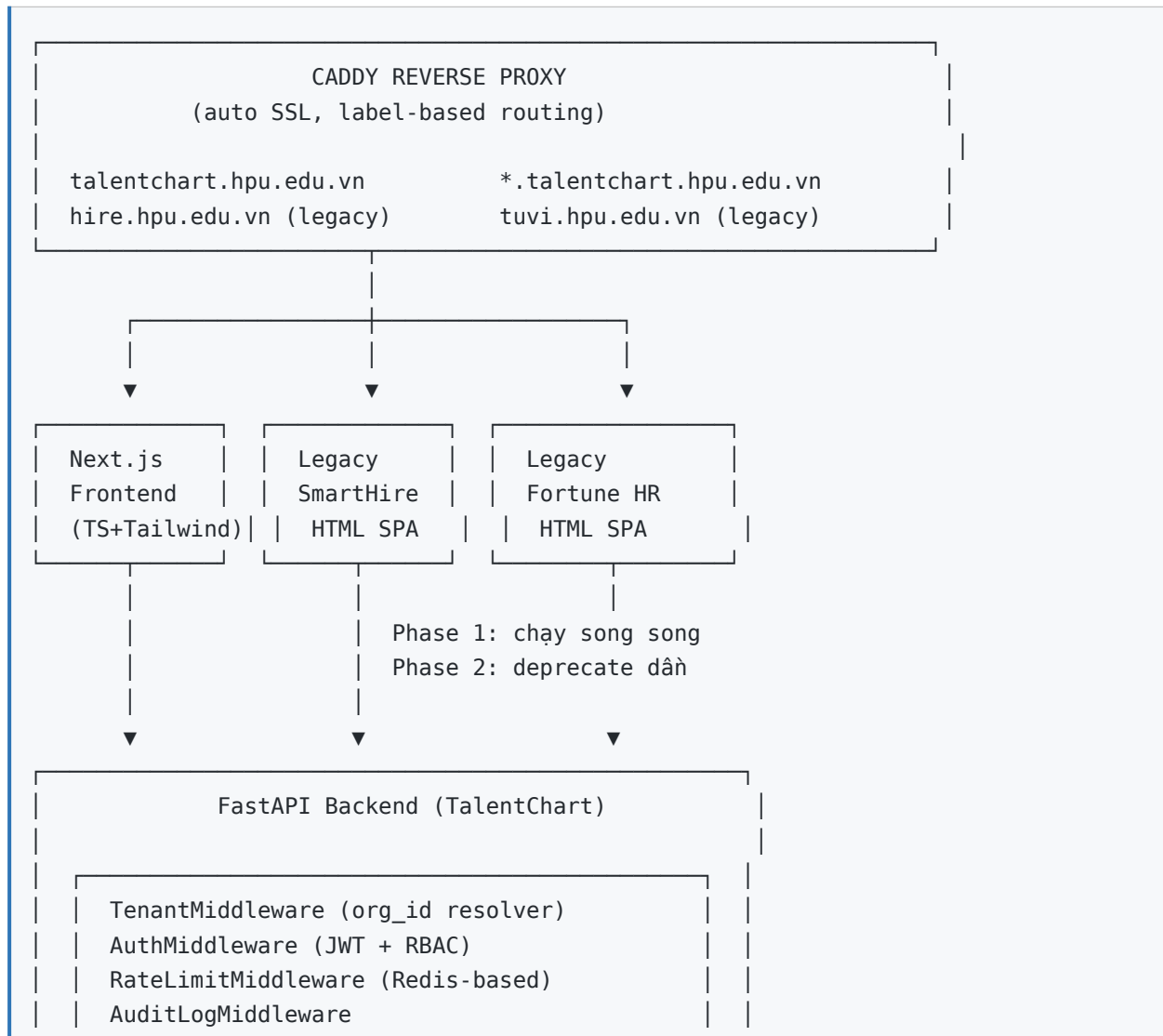
3.1. Sơ đồ component

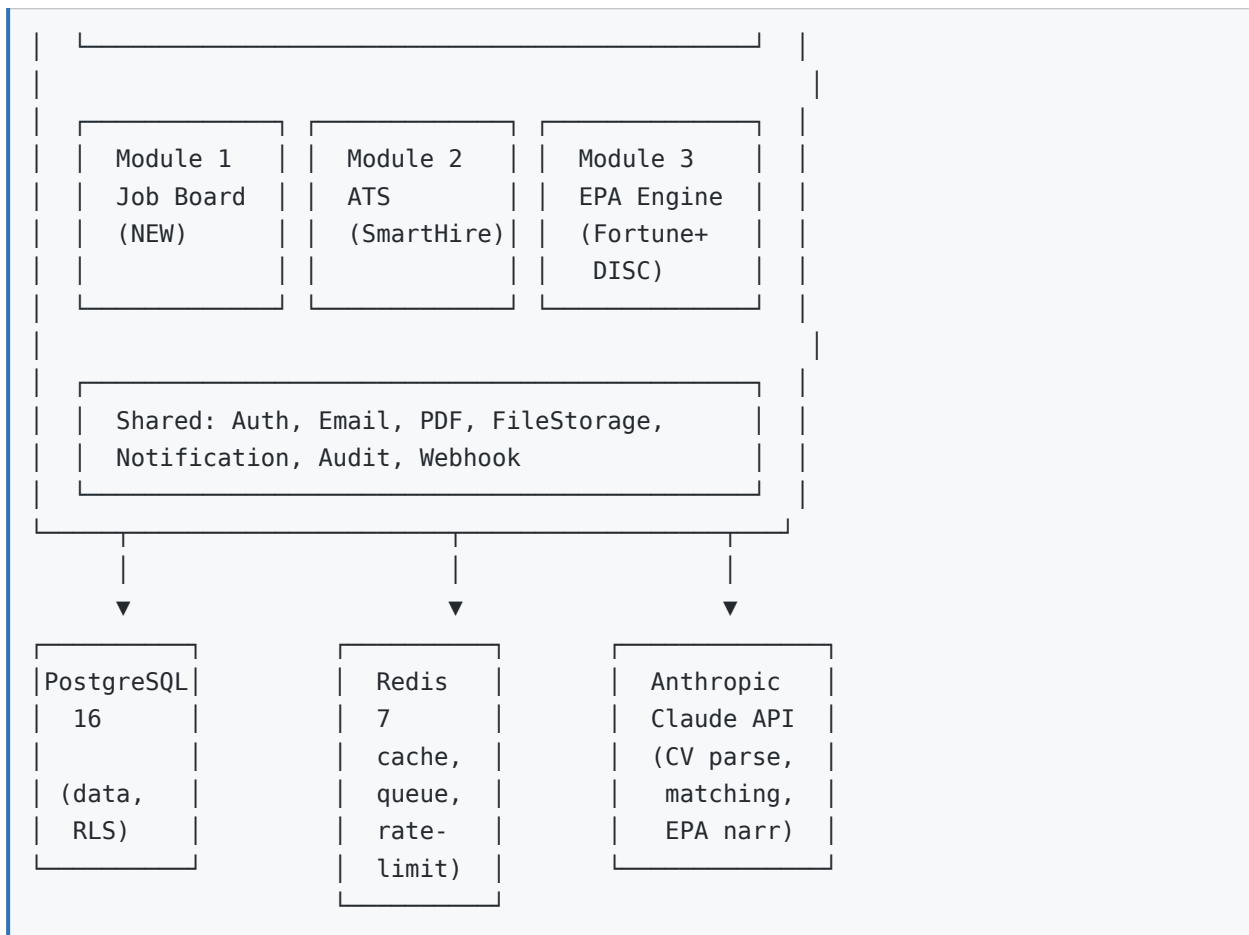
TalentChart Phase 1 là monolithic FastAPI application với module hóa rõ ràng. Không phải microservices — microservices ở Phase 1 sẽ giết tốc độ phát triển. Khi nào MRR đạt 500tr (Phase 3-4), mới cân nhắc tách microservices nếu cần.

Quyết định: Modular Monolith, không Microservices

Lý do: 1 mid dev + 2 intern không thể vận hành microservices đúng chuẩn (service discovery, distributed tracing, network latency, eventual consistency). Modular monolith cho hiệu năng dev cao gấp 3-5 lần ở giai đoạn này, có thể tách microservices sau khi mỗi module trưởng thành rõ ràng.

Sơ đồ 4 lớp





3.2. Tech stack đầy đủ với version cụ thể

Lựa chọn version dựa trên SmartHire hiện có + cập nhật những version stable mới nhất tháng 4/2026:

Backend

Component	Tech	Version	Lý do
Language	Python	3.12	Performance tốt, async đầy đủ
Web framework	FastAPI	0.115+	Đã dùng trong SmartHire, async chuẩn
ASGI server	Uvicorn	0.34+	Standard cho FastAPI
ORM	SQLAlchemy	2.0+	Async, đã dùng SmartHire
Migration	Alembic	1.14+	Đã dùng SmartHire
Validation	Pydantic	2.10+	v2 nhanh hơn v1 5-50x
DB driver async	asyncpg	0.30+	Driver Postgres nhanh nhất
DB driver	psycopg2-binary	2.9+	Cho Alembic

Component	Tech	Version	Lý do
sync			
Auth	python-jose + bcrypt	3.3 + 4.2	JWT + password hash
Email	aiosmtplib + jinja2	3.0 + 3.1	Async SMTP + template
PDF	ReportLab	4.2+	Đã dùng SmartHire
HTTP client	httpx	0.28+	Async, gọi Anthropic API
Rate limit	slowapi	0.1.9+	FastAPI native
Logging	structlog	24.4+	Structured logging
Testing	pytest + pytest-asyncio	8.3 + 0.24	Standard

Database

Component	Tech	Version	Mục đích
Primary DB	PostgreSQL	16-alpine	Dữ liệu chính, có Row Level Security
Cache + queue	Redis	7-alpine	Cache session, rate limit, background queue
File storage	MinIO	latest	S3-compatible, lưu CV, avatar (Phase 1 đơn giản, Phase 2 có thể chuyển AWS S3)
Vector DB (Phase 2)	pgvector extension	0.7+	AI matching CV-JD semantic search

Frontend

Component	Tech	Version	Lý do
Framework	Next.js	15+	App Router, RSC, dễ SEO cho Job Board
Language	TypeScript	5.6+	Type safety bắt buộc cho team junior
UI library	Tailwind CSS	3.4+	Utility-first, vibe-coding-friendly
Component lib	shadcn/ui	latest	Copy-paste components, dễ tùy chỉnh
State	Zustand + TanStack Query	5+ / 5+	Đơn giản, không Redux phức tạp
Form	React Hook Form + Zod	7+ / 3+	Validation đồng bộ với Pydantic backend
Icons	Lucide React	latest	Đẹp, light

Component	Tech	Version	Lý do
Date/time	date-fns	4+	Locale Việt Nam tốt

DevOps & Tooling

Component	Tech	Mục đích
Container	Docker + Docker Compose	Build, deploy, dev environment
Reverse proxy	Caddy 2	Auto SSL, label routing (đã có)
CI/CD	GitHub Actions	Test, lint, build trên mỗi PR
Code quality	ruff + mypy + pre-commit	Format, lint, type check
Frontend lint	ESLint + Prettier + Biome	Format, lint TypeScript
Monitoring	Uptime Kuma + Grafana (self-host)	Uptime + metrics
Error tracking	Sentry self-hosted (Phase 2)	Production error tracking
Backup	pg_dump + crontab + rsync	Backup DB tự động, lưu trên server riêng

3.3. Cấu trúc thư mục monorepo

Toàn bộ TalentChart trong 1 monorepo Git. Cấu trúc thư mục được thiết kế để vibe coding: dev và Cursor đều dễ navigate.

```
talentchart/
├─ README.md
├─ .env.example
├─ .gitignore
├─ .editorconfig
├─ docker-compose.yml          # Production
├─ docker-compose.dev.yml      # Development override
├─ docker-compose.legacy.yml   # Run hire/tuvi old containers in parallel
├─ Caddyfile                   # Reverse proxy config
├─
├─ docs/
│  ├─ architecture.md          # File này
│  ├─ playbook.md              # Vibe coding playbook
│  ├─ adr/                      # Architecture Decision Records
│  │  ├─ 001-multi-tenant-strategy.md
│  │  ├─ 002-monolith-vs-microservices.md
│  │  └─ 003-frontend-framework.md
│  └─ api/                     # OpenAPI specs (auto-generated)
├─
```

```

└─ backend/
  └─ Dockerfile
  └─ pyproject.toml      # Python deps with poetry/uv
  └─ alembic.ini
  └─ alembic/
    └─ versions/        # All migrations
  └─ app/
    └─ main.py          # FastAPI app
    └─ config.py        # Settings (fail-fast)
    └─ database.py      # Async session
    └─ exceptions.py    # Custom exceptions
    └─ middleware/
      └─ tenant.py      # ★ Multi-tenant middleware
      └─ auth.py
      └─ rate_limit.py
      └─ audit.py
    └─ core/
      └─ security.py    # JWT, password hashing
      └─ permissions.py # RBAC checks
      └─ tenant_context.py # Current org context
    └─ models/          # SQLAlchemy models
      └─ __init__.py
      └─ base.py        # ★ TenantScopedBase
      └─ organization.py # ★ NEW
      └─ user.py
      └─ candidate.py
      └─ campaign.py
      └─ interview.py
      └─ job_post.py    # ★ NEW (Module 1)
      └─ epa_profile.py # ★ NEW (Module 3)
      └─ audit_log.py   # NEW
    └─ schemas/         # Pydantic schemas (mirror models)
    └─ routers/         # API endpoints
      └─ __init__.py
      └─ auth.py
      └─ organizations.py # NEW
      └─ users.py        # NEW
      └─ candidates.py
      └─ campaigns.py
      └─ interviews.py
      └─ test_links.py
      └─ job_posts.py   # ★ NEW
      └─ careers.py     # ★ NEW (public job page)
      └─ epa.py         # ★ NEW
      └─ public.py      # NEW (career page, B2C bất tự)
    └─ services/        # Business logic
      └─ disc.py        # From SmartHire

```


4. MULTI-TENANT — TRÁI TIM CỦA PHASE 1

ĐÂY LÀ PHẦN QUAN TRỌNG NHẤT CỦA TÀI LIỆU

Mọi dev tham gia dự án PHẢI đọc kỹ chương 4 này. Một lỗi multi-tenant có thể làm rò rỉ dữ liệu giữa các trường khách hàng và giết startup ngay lập tức.

Nếu chưa hiểu phần nào, hỏi Trung trước khi viết code.

4.1. Mô hình tenant lựa chọn

Có 3 mô hình multi-tenancy phổ biến cho SaaS:

Mô hình	Mô tả	Ưu điểm	Nhược điểm
Database per tenant	Mỗi customer 1 database riêng	Isolation tuyệt đối, dễ backup riêng	Phức tạp vận hành, đắt khi nhiều khách
Schema per tenant	1 DB nhưng mỗi customer 1 schema	Cách ly tốt, query đơn giản	Khó migration đồng bộ, nhiều schema
Shared schema (pooled)	Tất cả customer chung schema, phân biệt bằng org_id	Đơn giản, hiệu quả cost, dễ scale	Risk data leak nếu code sai

Quyết định: Shared Schema (Pooled) với Row Level Security (RLS)

Lý do:

1. Phù hợp với 50-200 customer trong Phase 1-2 (mục tiêu cuối Phase 2 là 50 paying)
2. PostgreSQL Row Level Security (RLS) là layer bảo vệ thứ 2 ở mức database, ngoài tenant filter ở application
3. Backup, restore, migration đơn giản
4. Chi phí infra thấp

Bù đắp nhược điểm bằng:

- PostgreSQL RLS bật cho mọi bảng tenant-scoped
- SQLAlchemy event listener auto-inject org filter
- Test isolation bắt buộc trong CI
- Audit log mọi truy cập cross-tenant (sẽ luôn là attempt unauthorized)

4.2. Database schema thay đổi

Bảng mới: organizations

```
CREATE TABLE organizations (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  slug        VARCHAR(50) UNIQUE NOT NULL,      -- 'hpu', 'truong-x'
  name        VARCHAR(255) NOT NULL,            -- 'Trường ĐH Quản lý và Công
nghệ HP'
  name_short  VARCHAR(50),                      -- 'HPU'
  type        VARCHAR(30) NOT NULL,             -- 'university', 'college',
'agency'
  tier         VARCHAR(30) DEFAULT 'starter',    -- 'starter', 'standard',
'premier'

  contact_email VARCHAR(255) NOT NULL,
  contact_phone VARCHAR(20),
  address      TEXT,
  website      VARCHAR(255),
  logo_url     VARCHAR(500),

  -- Subdomain config
  subdomain    VARCHAR(50) UNIQUE,              -- 'hpu' →
hpu.talentchart.hpu.edu.vn
  custom_domain VARCHAR(255) UNIQUE,            -- 'careers.hpu.edu.vn' (Phase
2)

  -- Settings
  settings     JSONB DEFAULT '{}',              -- branding, EPA toggle, etc
  features     JSONB DEFAULT '{}',              -- enabled features per tier

  -- Status
  is_active    BOOLEAN DEFAULT TRUE,
  suspended_at TIMESTAMPTZ,
  trial_ends_at TIMESTAMPTZ,

  -- Audit
  created_at   TIMESTAMPTZ DEFAULT NOW(),
  updated_at   TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_organizations_slug ON organizations(slug);
CREATE INDEX idx_organizations_subdomain ON organizations(subdomain);
CREATE INDEX idx_organizations_active ON organizations(is_active) WHERE is_active =
TRUE;
```

Sửa bảng users — thêm organization scope

```
-- Migration: thêm organization_id và org-level role
ALTER TABLE users
  ADD COLUMN organization_id UUID NOT NULL REFERENCES organizations(id),
  ADD COLUMN org_role VARCHAR(30) NOT NULL DEFAULT 'member',
  ADD COLUMN must_change_password BOOLEAN DEFAULT TRUE;

-- Index cho query theo org
CREATE INDEX idx_users_organization_id ON users(organization_id);

-- Unique constraint: username unique trong CÙNG 1 org (không phải global)
ALTER TABLE users DROP CONSTRAINT users_username_key;
ALTER TABLE users DROP CONSTRAINT users_email_key;
CREATE UNIQUE INDEX uq_users_org_username ON users(organization_id, username);
CREATE UNIQUE INDEX uq_users_org_email ON users(organization_id, email);

-- Bật Row Level Security
ALTER TABLE users ENABLE ROW LEVEL SECURITY;

-- Policy: chỉ thấy user trong cùng org của mình
CREATE POLICY users_isolation ON users
  USING (organization_id = current_setting('app.current_org_id')::UUID);
```

Thêm org_id vào tất cả bảng tenant-scoped

Pattern này áp dụng cho candidates, campaigns, interview_scores, test_links, job_posts, epa_profiles...

```
-- Mẫu cho bảng candidates (áp dụng tương tự cho mọi bảng)
ALTER TABLE candidates
  ADD COLUMN organization_id UUID NOT NULL REFERENCES organizations(id);

CREATE INDEX idx_candidates_org_id ON candidates(organization_id);
CREATE INDEX idx_candidates_org_status ON candidates(organization_id, status);

-- Email không còn unique global – unique trong cùng org
DROP INDEX IF EXISTS ix_candidates_email;
CREATE UNIQUE INDEX uq_candidates_org_email ON candidates(organization_id, email);

-- Bật RLS
ALTER TABLE candidates ENABLE ROW LEVEL SECURITY;
CREATE POLICY candidates_isolation ON candidates
  USING (organization_id = current_setting('app.current_org_id')::UUID);
```

TenantScopedBase — Base class cho models

```
# backend/app/models/base.py
```

```

import uuid
from sqlalchemy import Column, ForeignKey, DateTime
from sqlalchemy.dialects.postgresql import UUID
from sqlalchemy.orm import declared_attr
from datetime import datetime, timezone
from app.database import Base

class TimestampMixin:
    created_at = Column(DateTime(timezone=True),
                        default=lambda: datetime.now(timezone.utc),
                        nullable=False)
    updated_at = Column(DateTime(timezone=True),
                        default=lambda: datetime.now(timezone.utc),
                        onupdate=lambda: datetime.now(timezone.utc),
                        nullable=False)

class TenantScopedBase(Base, TimestampMixin):
    """
    Base class cho mọi model thuộc về một tenant.
    Tự động thêm organization_id NOT NULL.

    LUẬT BẮT BUỘC:
    - Mọi model dữ liệu khách hàng PHẢI inherit từ class này
    - Mọi query trên model này PHẢI có tenant context (set qua middleware)
    """
    __abstract__ = True

    id = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)

    @declared_attr
    def organization_id(cls):
        return Column(
            UUID(as_uuid=True),
            ForeignKey("organizations.id", ondelete="CASCADE"),
            nullable=False,
            index=True
        )

```

4.3. Middleware tenant context

Đây là cốt lõi của multi-tenancy. Middleware này chạy đầu tiên trong mọi request, set context để mọi query phía sau tự động filter theo tenant đúng.

```
# backend/app/middleware/tenant.py

from fastapi import Request, HTTPException
from starlette.middleware.base import BaseHTTPMiddleware
from sqlalchemy import text
from app.database import async_session_maker
from app.core.tenant_context import set_tenant_context, clear_tenant_context

class TenantMiddleware(BaseHTTPMiddleware):
    """
    Resolve tenant từ:
    1. JWT token (cho API authenticated)
    2. Subdomain (cho public pages, ví dụ careers.hpu.talentchart.hpu.edu.vn)
    3. Header X-Organization-Slug (cho dev/testing)

    Set context cho session DB toàn request.
    """

    PUBLIC_PATHS = {"/api/v1/health", "/api/v1/auth/login", "/docs",
"/openapi.json"}

    async def dispatch(self, request: Request, call_next):
        # Skip cho public paths
        if any(request.url.path.startswith(p) for p in self.PUBLIC_PATHS):
            return await call_next(request)

        org_id = await self._resolve_org_id(request)
        if not org_id:
            raise HTTPException(403, "Tenant context required")

        # Set context cho cả Python (contextvar) và Postgres session
        token = set_tenant_context(org_id)
        try:
            # Set Postgres session variable cho RLS
            async with async_session_maker() as session:
                await session.execute(
                    text(f"SET LOCAL app.current_org_id = '{org_id}'")
                )
            request.state.db = session
            request.state.org_id = org_id
            response = await call_next(request)
            return response
        finally:
            clear_tenant_context(token)

    async def _resolve_org_id(self, request: Request):
```

```

# Strategy 1: từ JWT token (đã decoded ở AuthMiddleware trước đó)
if hasattr(request.state, "user"):
    return request.state.user.organization_id

# Strategy 2: từ subdomain (cho public pages)
host = request.headers.get("host", "")
if "." in host:
    subdomain = host.split(".")[0]
    if subdomain not in {"www", "api", "talentchart"}:
        # Lookup org by subdomain
        async with async_session_maker() as session:
            org = await get_org_by_subdomain(session, subdomain)
            if org:
                return str(org.id)

# Strategy 3: từ header (dev only)
if settings.ENV == "development":
    slug = request.headers.get("X-Organization-Slug")
    if slug:
        async with async_session_maker() as session:
            org = await get_org_by_slug(session, slug)
            if org:
                return str(org.id)

return None

```

Tenant Context (Python contextvar)

```

# backend/app/core/tenant_context.py

from contextvars import ContextVar
from uuid import UUID
from typing import Optional

_current_org_id: ContextVar[Optional[UUID]] = ContextVar(
    "current_org_id",
    default=None
)

def set_tenant_context(org_id: str | UUID):
    """Called by TenantMiddleware at request start."""
    if isinstance(org_id, str):
        org_id = UUID(org_id)
    return _current_org_id.set(org_id)

```

```
def get_current_org_id() -> Optional[UUID]:
    """Available anywhere in the request lifecycle."""
    return _current_org_id.get()

def get_current_org_id_required() -> UUID:
    """Use when org context MUST be present."""
    org_id = _current_org_id.get()
    if org_id is None:
        raise RuntimeError(
            "No tenant context – middleware not applied or path is public?"
        )
    return org_id

def clear_tenant_context(token):
    _current_org_id.reset(token)
```

SQLAlchemy event listener — auto-filter every query

```
# backend/app/database.py (additions)

from sqlalchemy import event
from sqlalchemy.orm import with_loader_criteria
from app.core.tenant_context import get_current_org_id
from app.models.base import TenantScopedBase

@event.listens_for(AsyncSession.sync_session, "do_orm_execute")
def _add_tenant_filter(execute_state):
    """
    BEZPIECZEŃSTWO KRYTYCZNE: tự động thêm organization_id filter
    vào MỌI query trên TenantScopedBase models.

    Code reviewer phải đảm bảo không ai disable cái này
    bằng cách dùng raw SQL.
    """
    if execute_state.is_select and not execute_state.is_relationship_load:
        org_id = get_current_org_id()
        if org_id is not None:
            execute_state.statement = execute_state.statement.options(
                with_loader_criteria(
                    TenantScopedBase,
                    lambda cls: cls.organization_id == org_id,
                    include_aliases=True
```

```
)
)
```

4.4. Subdomain routing

Mỗi tổ chức có subdomain riêng. URL pattern:

- Main app cho admin: app.talentchart.hpu.edu.vn (tất cả org đăng nhập tại đây)
- Public career page: {org-slug}.talentchart.hpu.edu.vn (ví dụ hpu.talentchart.hpu.edu.vn cho HPU)
- Phase 2 hỗ trợ custom domain: careers.hpu.edu.vn (ví dụ HPU dùng tên miền riêng)

Caddyfile config

```
# Caddyfile

# Wildcard cho mọi subdomain (cần wildcard SSL cert)
*.talentchart.hpu.edu.vn {
    tls {
        dns cloudflare {env.CLOUDFLARE_API_TOKEN}
    }

    # Route tất cả về Next.js, Next.js sẽ check subdomain để render đúng page
    reverse_proxy talentchart-frontend:3000

    # API calls cũng đi qua frontend (Next.js proxy)
    handle /api/* {
        reverse_proxy talentchart-backend:8083
    }
}

# Main domain
talentchart.hpu.edu.vn {
    redir https://app.talentchart.hpu.edu.vn{uri}
}

app.talentchart.hpu.edu.vn {
    reverse_proxy talentchart-frontend:3000
    handle /api/* {
        reverse_proxy talentchart-backend:8083
    }
}

# Legacy – chạy song song trong Phase 1
hire.hpu.edu.vn {
    reverse_proxy smarthire-legacy:8083
}
```

```
tuvi.hpu.edu.vn {  
  reverse_proxy fortune-hr-legacy:80  
}
```

Next.js subdomain handler

```
// frontend/src/middleware.ts  
  
import { NextRequest, NextResponse } from 'next/server';  
  
export function middleware(req: NextRequest) {  
  const host = req.headers.get('host') || '';  
  const subdomain = host.split('.')[0];  
  
  const ADMIN_SUBDOMAINS = ['app', 'www', 'talentchart'];  
  
  // Subdomain là org slug → public career page  
  if (!ADMIN_SUBDOMAINS.includes(subdomain) &&  
    !host.startsWith('localhost')) {  
    // Rewrite tới /careers/[org] route  
    const url = req.nextUrl.clone();  
    url.pathname = `/careers/${subdomain}${url.pathname}`;  
    return NextResponse.rewrite(url);  
  }  
  
  return NextResponse.next();  
}
```

5. DATABASE SCHEMA CHI TIẾT

Phần này định nghĩa toàn bộ tables của TalentChart Phase 1. Tất cả tables tenant-scoped đều inherit từ TenantScopedBase (organization_id NOT NULL + RLS enabled).

5.1. Tổng quan các bảng

Bảng	Mô tả	Tenant-scoped?	Nguồn
organizations	Tổ chức khách hàng	✗ (root)	MỚI
users	Tài khoản đăng nhập	✓	SmartHire — refactor
candidates	Ứng viên (HR) hoặc cán bộ (Fortune)	✓	SmartHire + Fortune merged
campaigns	Đợt tuyển dụng	✓	SmartHire
job_posts	Tin tuyển dụng public	✓	MỚI (Module 1)
test_links	Token gửi test cho ứng viên	✓	SmartHire
interview_scores	Điểm phỏng vấn	✓	SmartHire
epa_profiles	Hồ sơ EPA của 1 candidate	✓	MỚI (Fortune fusion)
epa_compatibility_cache	Cache kết quả tương hợp	✓	MỚI (perf)
audit_logs	Lịch sử hành động	✓	MỚI (compliance)
files	Metadata file upload	✓	MỚI

5.2. Schema chi tiết cho từng bảng

organizations (root, không tenant-scoped)

Đã định nghĩa ở mục 4.2. Đây là bảng root duy nhất.

users — đã refactor multi-tenant

```
CREATE TABLE users (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  organization_id   UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,

  username          VARCHAR(100) NOT NULL,
  email             VARCHAR(255) NOT NULL,
  full_name         VARCHAR(255) NOT NULL,
```

```

        hashed_password          VARCHAR(255) NOT NULL,

        -- 2 lớp role
        role                     VARCHAR(30) NOT NULL,          -- legacy from SmartHire
        org_role                  VARCHAR(30) NOT NULL DEFAULT 'member', -- 'owner',
        'admin', 'member'

        -- Profile
        avatar_url                VARCHAR(500),
        phone                     VARCHAR(20),

        -- Status
        is_active                  BOOLEAN DEFAULT TRUE,
        must_change_password       BOOLEAN DEFAULT TRUE,
        last_login_at              TIMESTAMPTZ,
        last_login_ip              INET,

        -- Audit
        created_at                 TIMESTAMPTZ DEFAULT NOW(),
        updated_at                 TIMESTAMPTZ DEFAULT NOW(),

        -- Tenant-scoped uniqueness
        CONSTRAINT uq_users_org_username UNIQUE(organization_id, username),
        CONSTRAINT uq_users_org_email UNIQUE(organization_id, email)
    );

ALTER TABLE users ENABLE ROW LEVEL SECURITY;
CREATE POLICY users_tenant_isolation ON users
    USING (organization_id = current_setting('app.current_org_id', TRUE)::UUID);

```

candidates — hợp nhất ứng viên HR + cán bộ Fortune

Đây là một quyết định kiến trúc quan trọng: Ứng viên (candidate) trong SmartHire và Cán bộ (employee) trong Fortune HR sẽ được hợp nhất thành 1 bảng candidates với enum candidate_type. Lý do: cả hai đều cần phân tích DISC + Bát tự + 12 archetype, dữ liệu giống nhau, không cần tách bảng.

```

CREATE TYPE candidate_type AS ENUM ('applicant', 'employee', 'student', 'alumni');

-- Pipeline status – 7 trạng thái (gọn hơn SmartHire 9 trạng thái cũ)
CREATE TYPE candidate_status AS ENUM (
    'NEW',          -- Vừa tiếp nhận
    'SCREENING',     -- Đang sàng lọc
    'TEST_SENT',    -- Đã gửi link test
    'TEST_DONE',    -- Đã hoàn thành test
    'INTERVIEW',    -- Đang phỏng vấn (có thể nhiều round, dùng interview_round)
    'DECISION',     -- Hội đồng quyết định
    'HIRED',        -- Đã tuyển

```

```

    'REJECTED'      -- Từ chối
);

CREATE TABLE candidates (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    organization_id    UUID NOT NULL REFERENCES organizations(id) ON DELETE
    CASCADE,

    -- Type discriminator
    candidate_type     candidate_type NOT NULL DEFAULT 'applicant',

    -- Personal info
    full_name          VARCHAR(255) NOT NULL,
    email              VARCHAR(255),
    phone              VARCHAR(20),

    -- For employees: organizational info
    employee_code       VARCHAR(50),          -- 'HPU001'
    department          VARCHAR(255),
    job_title           VARCHAR(255),
    is_family_member    BOOLEAN DEFAULT FALSE, -- Fortune HR feature

    -- For applicants: position info
    position            VARCHAR(300),
    education           VARCHAR(100),
    experience_years     INTEGER DEFAULT 0,

    -- Birth data (for EPA Engine)
    birth_day           SMALLINT,             -- 1-31
    birth_month         SMALLINT,             -- 1-12
    birth_year          SMALLINT,             -- 1900-2100
    birth_hour          SMALLINT,             -- 0-23, optional
    birth_place         VARCHAR(255),         -- optional, future timezone calc

    -- CV
    cv_url              VARCHAR(500),         -- file storage URL
    cv_text             TEXT,                 -- parsed text for AI search

    -- Status (only for applicants) – 7 trạng thái
    campaign_id         UUID REFERENCES campaigns(id),
    status              candidate_status DEFAULT 'NEW',

    -- DISC results
    disc_scores         JSONB,                -- {D: 30, I: 25, S: 20, C: 25}
    disc_primary         VARCHAR(5),
    disc_secondary       VARCHAR(5),
    disc_profile         VARCHAR(10),

```

```

disc_completed_at      TIMESTAMPTZ,

-- Personality results
personality_scores     JSONB,

-- AI analysis
ai_match_score         NUMERIC(5,2),
ai_analysis            JSONB,
ai_interview_suggestions JSONB,

-- Overall
overall_score          NUMERIC(5,2),
recommendation         VARCHAR(50),

-- Notes
notes                 TEXT,
tags                 TEXT[],          -- ['urgent', 'top-pick', ...]

-- Audit
created_at            TIMESTAMPTZ DEFAULT NOW(),
updated_at           TIMESTAMPTZ DEFAULT NOW(),

CONSTRAINT uq_candidates_org_email UNIQUE(organization_id, email),
CONSTRAINT uq_candidates_org_emp_code UNIQUE(organization_id, employee_code)
);

CREATE INDEX idx_candidates_org_status ON candidates(organization_id, status);
CREATE INDEX idx_candidates_org_type ON candidates(organization_id,
candidate_type);
CREATE INDEX idx_candidates_org_dept ON candidates(organization_id, department)
WHERE candidate_type = 'employee';

ALTER TABLE candidates ENABLE ROW LEVEL SECURITY;
CREATE POLICY candidates_tenant_isolation ON candidates
USING (organization_id = current_setting('app.current_org_id', TRUE)::UUID);

```

epa_profiles — Module 3 USP

```

CREATE TABLE epa_profiles (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  organization_id   UUID NOT NULL REFERENCES organizations(id) ON DELETE
CASCADE,
  candidate_id      UUID NOT NULL REFERENCES candidates(id) ON DELETE
CASCADE,

-- Bất tự core

```

```

    lunar_year          SMALLINT NOT NULL,
    lunar_month         SMALLINT NOT NULL,
    lunar_day           SMALLINT NOT NULL,

    thien_can           VARCHAR(10) NOT NULL,    -- 'Giáp', 'Ât'...
    dia_chi              VARCHAR(10) NOT NULL,    -- 'Tý', 'Sử'...
    con_giap             VARCHAR(20) NOT NULL,    -- 'Chuột', 'Trâu'...
    nap_am              VARCHAR(50) NOT NULL,    -- 'Hải Trung Kim'...
    menh                VARCHAR(20) NOT NULL,    -- 'Kim', 'Mộc', 'Thủy',
    'Hỏa', 'Thô'

    -- 12 Archetype (USP fusion)
    archetype_code       VARCHAR(30) NOT NULL,    -- 'STRATEGIST', 'BUILDER'...
    archetype_score      NUMERIC(5,2),          -- confidence 0-100
    archetype_secondary  VARCHAR(30),

    -- Component scores
    disc_input           JSONB NOT NULL,
    epa_input            JSONB NOT NULL,

    -- AI narrative
    narrative_short      TEXT,
    narrative_full       TEXT,
    strengths            TEXT[],
    weaknesses           TEXT[],
    suitable_roles       TEXT[],
    suitable_partners    JSONB,                  -- {primary: [...], avoid:
[...]}

    -- Versioning (when archetype algorithm updates)
    algorithm_version    VARCHAR(20) NOT NULL DEFAULT '1.0',

    created_at           TIMESTAMPTZ DEFAULT NOW(),
    updated_at           TIMESTAMPTZ DEFAULT NOW(),

    CONSTRAINT uq_epa_candidate UNIQUE(candidate_id, algorithm_version)
);

CREATE INDEX idx_epa_org_archetype ON epa_profiles(organization_id,
archetype_code);
CREATE INDEX idx_epa_org_menh ON epa_profiles(organization_id, menh);

ALTER TABLE epa_profiles ENABLE ROW LEVEL SECURITY;
CREATE POLICY epa_tenant_isolation ON epa_profiles
    USING (organization_id = current_setting('app.current_org_id', TRUE)::UUID);

```

job_posts — Module 1 mới

```

CREATE TYPE job_status AS ENUM ('draft', 'published', 'closed', 'archived');

CREATE TABLE job_posts (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    organization_id   UUID NOT NULL REFERENCES organizations(id) ON DELETE
    CASCADE,

    -- Slug for public URL
    slug              VARCHAR(255) NOT NULL,

    title              VARCHAR(500) NOT NULL,
    department         VARCHAR(255),
    job_type           VARCHAR(50),          -- 'full-time', 'part-time'
    location           VARCHAR(255),
    salary_min         NUMERIC(15,2),
    salary_max         NUMERIC(15,2),
    salary_visible     BOOLEAN DEFAULT FALSE,

    description        TEXT NOT NULL,
    requirements       TEXT,
    benefits           TEXT,

    -- Application config
    apply_method       VARCHAR(20) DEFAULT 'form', -- 'form', 'email',
    'external'
    external_url       VARCHAR(500),
    apply_email        VARCHAR(255),
    require_disc_test  BOOLEAN DEFAULT FALSE,
    require_epa        BOOLEAN DEFAULT FALSE,

    -- Link to campaign (optional)
    campaign_id        UUID REFERENCES campaigns(id),

    -- Status & timing
    status             job_status DEFAULT 'draft',
    published_at       TIMESTAMPTZ,
    expires_at         TIMESTAMPTZ,

    -- Stats
    view_count         INTEGER DEFAULT 0,
    apply_count        INTEGER DEFAULT 0,

    -- SEO
    meta_description   TEXT,

    created_by         UUID REFERENCES users(id),

```

```

        created_at          TIMESTAMPTZ DEFAULT NOW(),
        updated_at          TIMESTAMPTZ DEFAULT NOW(),

        CONSTRAINT uq_job_posts_org_slug UNIQUE(organization_id, slug)
    );

CREATE INDEX idx_job_posts_org_status ON job_posts(organization_id, status);
CREATE INDEX idx_job_posts_published ON job_posts(status, published_at DESC)
    WHERE status = 'published';

ALTER TABLE job_posts ENABLE ROW LEVEL SECURITY;
CREATE POLICY job_posts_tenant_isolation ON job_posts
    USING (organization_id = current_setting('app.current_org_id', TRUE)::UUID);

```

audit_logs — compliance

```

CREATE TABLE audit_logs (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    organization_id    UUID NOT NULL REFERENCES organizations(id) ON DELETE
    CASCADE,

    user_id            UUID REFERENCES users(id),
    user_email         VARCHAR(255),          -- denormalized in case user
    deleted

    action             VARCHAR(50) NOT NULL,  -- 'create', 'update',
    'delete', 'view', 'login', 'export'
    resource_type       VARCHAR(50) NOT NULL,  -- 'candidate', 'campaign',
    'epa_profile'
    resource_id         UUID,

    -- Change details
    changes             JSONB,                  -- {field: {from, to}}
    metadata            JSONB,                  -- request context

    -- Request context
    ip_address          INET,
    user_agent          TEXT,

    created_at          TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_audit_org_created ON audit_logs(organization_id, created_at DESC);
CREATE INDEX idx_audit_org_user ON audit_logs(organization_id, user_id, created_at
DESC);
CREATE INDEX idx_audit_resource ON audit_logs(organization_id, resource_type,

```

```
resource_id);

-- audit_logs có RLS nhưng chỉ admin xem được
ALTER TABLE audit_logs ENABLE ROW LEVEL SECURITY;
CREATE POLICY audit_tenant_isolation ON audit_logs
    USING (organization_id = current_setting('app.current_org_id', TRUE)::UUID);
```

6. EPA ENGINE — USP CỐT LÕI

Đây là module tạo ra giá trị thương mại lớn nhất của TalentChart

Không có module này, TalentChart chỉ là một ATS bình thường cạnh tranh với Base.vn, MISA. Có module này, TalentChart là sản phẩm duy nhất tại VN, không có đối thủ trực tiếp.

6.1. Port Fortune HR sang Python

Engine bắt tự của Fortune HR được port nguyên xi từ JavaScript sang Python, KHÔNG cải tiến gì ở Phase 1. Lý do: thuật toán đã chạy production ổn định, mọi thay đổi có thể introduce bug.

Mapping JavaScript → Python

Fortune HR (JS)	TalentChart (Python)	Vị trí mới
LunarData array	LUNAR_DATA: list[int]	services/epa/lunar.py
getLunarYearDays(y)	get_lunar_year_days(y)	services/epa/lunar.py
getLeapMonth(y)	get_leap_month(y)	services/epa/lunar.py
getLunarDate(y, m, d)	get_lunar_date(y, m, d)	services/epa/lunar.py
getCanChiFromBirthdate	get_canchi_from_birth	services/epa/canchi.py
NAP_AM constants	NAP_AM_60	services/epa/canchi.py
MENH_MAP	MENH_BY_NAP_AM	services/epa/menh.py
TAM_HOP, XUNG	TAM_HOP, XUNG	services/epa/tamhop.py
/api/team-suggest logic	TeamChemistryService	services/epa/team_chemistry.py
/api/compatibility logic	CompatibilityService	services/epa/compatibility.py

Ví dụ port: lunar.py

```
# backend/app/services/epa/lunar.py

"""
Engine lịch âm Việt Nam, port từ Fortune HR (Express.js) sang Python.
Độ chính xác: 1900-2100.
KHÔNG sửa logic – đã được kiểm chứng production tại tuvi.hpu.edu.vn.
"""

from typing import TypedDict
from datetime import date
```

```

# Hex-encoded lunar data 1900-2100 (200 years)
# Mỗi entry encode: số ngày các tháng + tháng nhuận
LUNAR_DATA = [
    0x04bd8, 0x04ae0, 0x0a570, 0x054d5, 0x0d260, 0x0d950, 0x16554,
    0x056a0, 0x09ad0, 0x055d2, 0x04ae0, 0x0a5b6, 0x0a4d0, 0x0d250,
    # ... (200 entries from Fortune HR)
]

class LunarDate(TypedDict):
    lunar_year: int
    lunar_month: int
    lunar_day: int
    is_leap_month: bool

def get_lunar_year_days(year: int) -> int:
    """Total days in a lunar year (including leap month)."""
    total = 348
    i = 0x8000
    while i > 0x8:
        if LUNAR_DATA[year - 1900] & i:
            total += 1
        i >>= 1
    return total + get_leap_days(year)

def get_leap_month(year: int) -> int:
    """Returns 0 if no leap month, else 1-12."""
    return LUNAR_DATA[year - 1900] & 0xf

def get_leap_days(year: int) -> int:
    """Days in leap month (29 or 30)."""
    if get_leap_month(year):
        return 30 if (LUNAR_DATA[year - 1900] & 0x10000) else 29
    return 0

def get_lunar_month_days(year: int, month: int) -> int:
    """Days in a regular lunar month."""
    return 30 if (LUNAR_DATA[year - 1900] & (0x10000 >> month)) else 29

def get_lunar_date(solar_year: int, solar_month: int, solar_day: int) -> LunarDate:

```

```

"""
Convert solar (Gregorian) date to lunar date.

Args:
    solar_year: 1900-2100
    solar_month: 1-12
    solar_day: 1-31

Returns:
    LunarDate with year, month, day, is_leap_month

Raises:
    ValueError: if year out of range
"""
if not (1900 <= solar_year <= 2100):
    raise ValueError(f"Year {solar_year} out of range (1900-2100)")

base = date(1900, 1, 31) # Lunar 1900-01-01
target = date(solar_year, solar_month, solar_day)
offset = (target - base).days

# Find lunar year
lunar_year = 1900
while lunar_year < 2100 and offset >= 0:
    days_in_year = get_lunar_year_days(lunar_year)
    if offset < days_in_year:
        break
    offset -= days_in_year
    lunar_year += 1

# Find lunar month
lunar_month = 1
leap_month = get_leap_month(lunar_year)
is_leap = False

i = 1
while i <= 12:
    if leap_month > 0 and i == leap_month + 1 and not is_leap:
        days_in_month = get_leap_days(lunar_year)
        is_leap = True
        i -= 1
    else:
        days_in_month = get_lunar_month_days(lunar_year, i)

    if offset < days_in_month:
        lunar_month = i
        break

```

```

        offset -= days_in_month
        i += 1

    return LunarDate(
        lunar_year=lunar_year,
        lunar_month=lunar_month,
        lunar_day=offset + 1,
        is_leap_month=is_leap
    )

```

Ví dụ port: canchi.py

```

# backend/app/services/epa/canchi.py

from typing import TypedDict
from app.services.epa.lunar import get_lunar_date

THIEN_CAN = ['Giáp', 'Ất', 'Bính', 'Đinh', 'Mậu',
             'Kỷ', 'Canh', 'Tân', 'Nhâm', 'Quý']

DIA_CHI = ['Tý', 'Sử', 'Dần', 'Mão', 'Thìn', 'Ty',
           'Ngọ', 'Mùi', 'Thân', 'Dậu', 'Tuất', 'Hợi']

CON_GIAP = ['Chuột', 'Trâu', 'Hô', 'Mèo', 'Rồng', 'Rắn',
            'Ngựa', 'Dê', 'Khỉ', 'Gà', 'Chó', 'Lợn']

NAP_AM_60 = [
    'Hải Trung Kim', 'Hải Trung Kim', 'Lư Trung Hỏa', 'Lư Trung Hỏa',
    'Đại Lâm Mộc', 'Đại Lâm Mộc', 'Lộ Bàng Thổ', 'Lộ Bàng Thổ',
    'Kiếm Phong Kim', 'Kiếm Phong Kim', 'Sơn Đầu Hỏa', 'Sơn Đầu Hỏa',
    # ... (60 entries from Fortune HR)
]

MENH_BY_NAP_AM = {
    'Kim': {'Hải Trung Kim', 'Kiếm Phong Kim', 'Bạch Lạp Kim',
            'Sa Trung Kim', 'Kim Bạch Kim', 'Thoa Xuyên Kim'},
    'Mộc': {'Đại Lâm Mộc', 'Dương Liễu Mộc', 'Tùng Bách Mộc',
            'Bình Địa Mộc', 'Tang Đô Mộc', 'Thạch Lưu Mộc'},
    'Thủy': {'Giản Hạ Thủy', 'Tuyền Trung Thủy', 'Trường Lưu Thủy',
            'Thiên Hà Thủy', 'Đại Khê Thủy', 'Đại Hải Thủy'},
    'Hỏa': {'Lư Trung Hỏa', 'Sơn Đầu Hỏa', 'Tích Lịch Hỏa',
            'Sơn Hạ Hỏa', 'Phúc Đăng Hỏa', 'Thiên Thượng Hỏa'},
    'Thổ': {'Lộ Bàng Thổ', 'Thành Đầu Thổ', 'Ôc Thượng Thổ',
            'Bích Thượng Thổ', 'Đại Trạch Thổ', 'Sa Trung Thổ'},
}

```

```

class CanChiResult(TypedDict):
    thien_can: str
    dia_chi: str
    con_giap: str
    tuoi_am: str          # 'Giáp Tý'
    nap_am: str           # 'Hải Trung Kim'
    menh: str             # 'Kim', 'Mộc', etc
    lunar_year: int
    lunar_month: int
    lunar_day: int

def get_canchi_from_birth(
    solar_day: int,
    solar_month: int,
    solar_year: int
) -> CanChiResult:
    """
    Tính Can Chi từ ngày sinh dương lịch.

    QUAN TRỌNG: dùng LUNAR YEAR, không phải SOLAR YEAR.
    Ví dụ: sinh 1/1/1938 dương → âm là Đinh Sửu (1937),
    KHÔNG phải Mậu Dần (1938). Bug fix từ Fortune HR.
    """
    lunar = get_lunar_date(solar_year, solar_month, solar_day)
    lunar_year = lunar['lunar_year']

    can_idx = (lunar_year - 4) % 10
    chi_idx = (lunar_year - 4) % 12
    nap_am_idx = (lunar_year - 4) % 60

    nap_am = NAP_AM_60[nap_am_idx]

    # Find mệnh from nap âm
    menh = 'Thô' # default fallback
    for element, nap_am_set in MENH_BY_NAP_AM.items():
        if nap_am in nap_am_set:
            menh = element
            break

    return CanChiResult(
        thien_can=THIEN_CAN[can_idx],
        dia_chi=DIA_CHI[chi_idx],
        con_giap=CON_GIAP[chi_idx],
        tuoi_am=f"{THIEN_CAN[can_idx]} {DIA_CHI[chi_idx]}",
        nap_am=nap_am,

```

```

    menh=menh,
    lunar_year=lunar_year,
    lunar_month=lunar['lunar_month'],
    lunar_day=lunar['lunar_day'],
)

```

6.2. Fusion DISC + Bát tự = 12 Archetype

Đây là phần CREATE NEW của TalentChart, không có trong Fortune HR hay SmartHire. Đây là USP thực sự — kết hợp 2 hệ tâm lý/văn hoá để tạo ra 12 mẫu hình đặc trưng cho người Việt.

Logic fusion

Công thức 12 Archetype

Archetype = f(DISC_profile, Mệnh_ngũ_hành, Tam_hợp_group)

Trong đó:

- DISC_profile (16 giá trị): D, I, S, C, D/I, D/C, I/D, I/S, I/C, S/I, S/C, C/D, C/S, C/I
- Mệnh_ngũ_hành (5 giá trị): Kim, Mộc, Thủy, Hỏa, Thổ
- Tam_hợp_group (4 giá trị): Thân-Tý-Thìn, Tỵ-Dậu-Sửu, Dần-Ngọ-Tuất, Hợi-Mão-Mùi

Tổng tổ hợp lý thuyết: $16 \times 5 \times 4 = 320$ combinations

Mapping về 12 archetype dựa trên đặc trưng nổi trội nhất.

12 Archetype lõi

Code	Tên (VI / EN)	Đặc trưng	Vị trí phù hợp
STRATEGIST	Chiến lược gia / Strategist	C+D, mệnh Kim/Thủy	CEO, COO, Director
VISIONARY	Tầm nhìn / Visionary	D+I, mệnh Hỏa/Mộc	Founder, CMO, Innovation
BUILDER	Kiến tạo / Builder	D+S, mệnh Thổ/Kim	Engineering Manager, PM
EXECUTOR	Thực thi / Executor	D+C, mệnh Kim/Thổ	Operations, Project lead
CONNECTOR	Kết nối / Connector	I+S, mệnh Mộc/Hỏa	Sales, BD, Account Mgr
CATALYST	Xúc tác / Catalyst	I+D, mệnh Hỏa/Mộc	Marketing, PR, Trainer

Code	Tên (VI / EN)	Đặc trưng	Vị trí phù hợp
HARMONIZER	Hòa giải / Harmonizer	S+I, mệnh Thủy/Mộc	HR, Customer Success
GUARDIAN	Bảo trợ / Guardian	S+C, mệnh Thổ/Kim	Admin, Compliance, QA
ANALYST	Phân tích / Analyst	C+S, mệnh Thủy/Kim	Data, Research, Finance
CRAFTSMAN	Thợ giỏi / Craftsman	C+I, mệnh Mộc/Thổ	Designer, Engineer, Specialist
MENTOR	Cố vấn / Mentor	S+D, mệnh Mộc/Thổ	Senior IC, Coach, Teacher
CHALLENGER	Đột phá / Challenger	D+I+ Mộc vượng	Sales hunter, Founder

Algorithm computation (đơn giản hoá)

```
# backend/app/services/epa/archetype.py

from typing import TypedDict
from app.services.epa.canchi import CanChiResult

ARCHETYPES = {
    'STRATEGIST': {
        'disc_pattern': ['C/D', 'D/C'],
        'menh_affinity': ['Kim', 'Thủy'],
        'description': '...',
    },
    'VISIONARY': {
        'disc_pattern': ['D/I', 'I/D'],
        'menh_affinity': ['Hỏa', 'Mộc'],
        'description': '...',
    },
    # ... 10 more
}

class ArchetypeResult(TypedDict):
    primary_code: str
    primary_score: float
    secondary_code: str | None
    secondary_score: float
    explanation: dict

def compute_archetype(
    disc_profile: str,
    disc_scores: dict[str, int],
    canchi: CanChiResult,
) -> ArchetypeResult:
    """
    Tính archetype từ DISC + Bát tự fusion.

    Scoring approach:
    - Mỗi archetype có điểm 0-100 dựa trên match DISC và mệnh
    - DISC weight: 60%, Mệnh weight: 30%, Tam hợp bonus: 10%
    - Trả về primary (cao nhất) và secondary (nếu cách primary <15đ)
    """
    scores = {}

    for code, archetype in ARCHETYPES.items():
        score = 0.0
```

```

# DISC match (0-60 points)
if disc_profile in archetype['disc_pattern']:
    score += 60
elif disc_profile[0] in [p[0] for p in archetype['disc_pattern']]:
    score += 35

# Mệnh affinity (0-30 points)
if canchi['menh'] in archetype['menh_affinity']:
    score += 30
elif _is_menh_compatible(canchi['menh'], archetype['menh_affinity']):
    score += 15

# Tam hợp bonus (0-10 points)
# ... (logic tương hợp)

scores[code] = score

sorted_scores = sorted(scores.items(), key=lambda x: x[1], reverse=True)
primary = sorted_scores[0]
secondary = sorted_scores[1] if (primary[1] - sorted_scores[1][1]) < 15 else
None

return ArchetypeResult(
    primary_code=primary[0],
    primary_score=primary[1],
    secondary_code=secondary[0] if secondary else None,
    secondary_score=secondary[1] if secondary else 0,
    explanation={
        'disc_contribution': '...',
        'menh_contribution': '...',
        'all_scores': scores,
    }
)

def _is_menh_compatible(menh1: str, menh_list: list[str]) -> bool:
    """Tương sinh: Kim→Thủy→Mộc→Hỏa→Thô→Kim"""
    sinh_chain = ['Kim', 'Thủy', 'Mộc', 'Hỏa', 'Thô']
    idx = sinh_chain.index(menh1)
    next_menh = sinh_chain[(idx + 1) % 5]
    return next_menh in menh_list

```

Narrative generation bằng AI

Sau khi tính xong archetype, dùng Claude API để sinh báo cáo dạng văn xuôi cá nhân hoá:

```
# backend/app/services/epa/narrative.py
```

```

from anthropic import AsyncAnthropic
from app.config import settings

client = AsyncAnthropic(api_key=settings.ANTHROPIC_API_KEY)

async def generate_narrative(
    archetype_code: str,
    canchi: dict,
    disc_scores: dict,
    candidate_name: str,
    organization_context: str = None,
) -> dict:
    """
    Sinh báo cáo EPA dạng văn xuôi bằng Claude.

    Output có 2 lớp:
    - Eastern Layer (cho B2C, SME tin văn hoá)
    - Behavioural Layer (cho B2B chuyên nghiệp)
    """

    prompt = f'''Bạn là chuyên gia tâm lý học tổ chức + cố vấn văn hoá Á Đông.

Hồ sơ ứng viên:
- Tên: {candidate_name}
- Mệnh: {canchi['menh']} ({canchi['nap_am']})
- Tuổi âm: {canchi['tuoi_am']}, con giáp {canchi['con_giap']}
- DISC profile: {disc_scores}
- Archetype được tính: {archetype_code}

Yêu cầu:
Viết báo cáo phân tích cá nhân, cấu trúc 2 lớp:

LỚP 1 - Phân tích hành vi (cho HR chuyên nghiệp):
- Phong cách làm việc đặc trưng (3-5 dòng)
- Điểm mạnh nổi bật (5 gạch đầu dòng)
- Điểm cần phát triển (3-4 gạch đầu dòng)
- Vị trí công việc phù hợp nhất

LỚP 2 - Diễn giải Á Đông (toggle on/off):
- Mệnh và đặc tính ngũ hành
- Quan hệ tương sinh tương khắc với người khác mệnh
- Lời khuyên về môi trường làm việc, đối tác

Tone: chuyên nghiệp, ám áp, không sáo rỗng.
Tránh ngôn ngữ bói toán hay dự đoán tương lai.

```

Output JSON format chính xác như sau:

```
{
  "behavioural": {
    "style_summary": "...",
    "strengths": [...],
    "growth_areas": [...],
    "ideal_roles": [...]
  },
  "eastern": {
    "menh_traits": "...",
    "compatibility": {
      "synergy_with": [...],
      "tension_with": [...]
    },
    "advice": "..."
  }
}

...

response = await client.messages.create(
    model="claude-sonnet-4-7",
    max_tokens=2000,
    messages=[{"role": "user", "content": prompt}]
)

return parse_json_response(response.content[0].text)
```

7. API DESIGN

API tuân theo REST principles + JSON:API conventions, có versioning rõ ràng để Phase 2 có thể thay đổi mà không break Phase 1 clients.

7.1. Nguyên tắc thiết kế API

- Versioning trong URL: `/api/v1/...` — khi cần breaking change Phase 2 thì `/api/v2/...`, giữ v1 chạy song song 6-12 tháng để client migrate.
- Resource-based URLs: `/api/v1/candidates/{id}` (đúng), không `/api/v1/getCandidate` (sai).
- HTTP methods chuẩn: GET (read), POST (create), PATCH (partial update), DELETE (delete). PUT chỉ dùng nếu replace hoàn toàn.
- Status codes chuẩn: 200 OK, 201 Created, 204 No Content, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 422 Validation Error, 500 Server Error.
- Response envelope: `{data: ..., meta: {...}, errors: [...]}` cho consistency.
- Pagination: `?page=1&page_size=20`, response include `{pagination: {total, page, page_size, total_pages}}`.
- Filter và sort: `?status=new&sort=-created_at` (- prefix = desc).
- Documentation tự động qua FastAPI OpenAPI tại `/docs`.

7.2. API Endpoints Phase 1 đầy đủ

Authentication & Organization

Method	Endpoint	Mô tả	Auth
POST	<code>/api/v1/auth/login</code>	Đăng nhập, trả JWT	Public
POST	<code>/api/v1/auth/refresh</code>	Refresh JWT	JWT
POST	<code>/api/v1/auth/logout</code>	Logout, blacklist JWT	JWT
POST	<code>/api/v1/auth/change-password</code>	Đổi mật khẩu	JWT
GET	<code>/api/v1/auth/me</code>	Thông tin user hiện tại	JWT
POST	<code>/api/v1/organizations</code>	Tạo org mới (super-admin)	Super
GET	<code>/api/v1/organizations/me</code>	Org của user hiện tại	JWT
PATCH	<code>/api/v1/organizations/me</code>	Sửa thông tin org	Owner
GET	<code>/api/v1/organizations/me/users</code>	Danh sách user trong org	Admin
POST	<code>/api/v1/organizations/me/users</code>	Thêm user vào org	Admin
GET	<code>/api/v1/organizations/me/audit-logs</code>	Audit logs của org	Admin

Candidates & Campaigns (giữ từ SmarHire + tenant filter)

Method	Endpoint	Mô tả	Auth
GET	/api/v1/candidates	Danh sách ứng viên (paginate)	HR Staff+
POST	/api/v1/candidates	Tạo ứng viên thủ công	HR Staff+
GET	/api/v1/candidates/{id}	Chi tiết ứng viên	HR Staff+
PATCH	/api/v1/candidates/{id}	Update ứng viên	HR Staff+
DELETE	/api/v1/candidates/{id}	Xoá ứng viên	HR Manager+
GET	/api/v1/candidates/{id}/pdf	Xuất PDF báo cáo	HR Staff+
POST	/api/v1/candidates/{id}/move	Chuyển trạng thái pipeline	HR Staff+
GET	/api/v1/campaigns	Danh sách đợt tuyển dụng	HR Staff+
POST	/api/v1/campaigns	Tạo đợt tuyển dụng mới	HR Manager+
GET	/api/v1/campaigns/{id}	Chi tiết campaign	HR Staff+
GET	/api/v1/campaigns/{id}/candidates	Ứng viên trong campaign	HR Staff+
GET	/api/v1/dashboard/stats	Thống kê tổng quan	HR Staff+

Job Posts (Module 1 mới)

Method	Endpoint	Mô tả	Auth
GET	/api/v1/job-posts	Danh sách jobs (admin view)	HR Staff+
POST	/api/v1/job-posts	Tạo job post mới	HR Manager+
GET	/api/v1/job-posts/{id}	Chi tiết job	HR Staff+
PATCH	/api/v1/job-posts/{id}	Cập nhật job	HR Manager+
POST	/api/v1/job-posts/{id}/publish	Publish job	HR Manager+
POST	/api/v1/job-posts/{id}/close	Close job	HR Manager+

Public Career Page (no auth, but tenant via subdomain)

Method	Endpoint	Mô tả
GET	/api/v1/careers/jobs	Danh sách job đang publish của org
GET	/api/v1/careers/jobs/{slug}	Chi tiết job by slug
POST	/api/v1/careers/jobs/{slug}/apply	Nộp đơn (anonymous)
GET	/api/v1/careers/about	Giới thiệu org

EPA Engine (Module 3 mới)

Method	Endpoint	Mô tả	Auth
POST	/api/v1/epa/calculate	Tính EPA từ ngày sinh	HR Staff+
GET	/api/v1/epa/profiles/{candidate_id}	Lấy profile EPA của candidate	HR Staff+
POST	/api/v1/epa/profiles/{candidate_id}/regenerate	Tính lại với version mới	HR Manager+
GET	/api/v1/epa/compatibility?id1=...&id2=...	Đo tương hợp 2 người	HR Staff+
POST	/api/v1/epa/team-suggest	Đề xuất đội nhóm	HR Staff+
GET	/api/v1/epa/lunar-date?d=&m=&y=	Convert dương → âm	Public
GET	/api/v1/epa/today	Can Chi hôm nay	Public
GET	/api/v1/epa/archetypes	Danh sách 12 archetype	Public

Test Submission & Interview

Method	Endpoint	Mô tả	Auth
POST	/api/v1/test-links	Tạo link test gửi candidate	HR Staff+
GET	/api/v1/test-links/verify/{token}	Verify token	Public
GET	/api/v1/test/questions	Lấy bộ câu hỏi (DISC + Personality)	Public/token
POST	/api/v1/test/submit	Nộp bài test	Public/token
GET	/api/v1/test/check-email	Check email đã test chưa	Public
POST	/api/v1/interviews/scores	Chấm điểm phỏng vấn	Interviewer+

Method	Endpoint	Mô tả	Auth
GET	/api/v1/interviews/scores/{candidate_id}	Xem điểm PV	HR Staff+

8. SECURITY & COMPLIANCE

Security là yêu cầu pháp lý + cạnh tranh

Nghị định 13/2023/NĐ-CP của Việt Nam quy định bảo vệ dữ liệu cá nhân — phạt đến 3% doanh thu nếu vi phạm. Trường ĐH công lập sẽ không mua sản phẩm không đáp ứng tiêu chuẩn bảo mật cơ bản.

8.1. Authentication & Authorization

JWT-based authentication

- JWT_SECRET phải là 256-bit random (sinh bằng openssl rand -hex 32), KHÔNG có default fallback — fail-fast nếu thiếu.
- Access token TTL: 15 phút. Refresh token TTL: 7 ngày, lưu trong Redis với revoke list.
- Token chứa: user_id, organization_id, roles, expiration. KHÔNG chứa password hay PII nhạy cảm.
- Rotate JWT_SECRET định kỳ 6 tháng (tự động grace period 7 ngày cho key cũ + mới).

Password handling

- Hash bằng bcrypt với cost factor 12 (kế thừa knowledge từ SmartHire).
- Force change password lần đầu đăng nhập (must_change_password flag).
- Password policy: tối thiểu 10 ký tự, có chữ hoa + chữ thường + số. Không yêu cầu ký tự đặc biệt (NIST 2024 guidance).
- Block common passwords (top 10K most-used) qua list check.
- Rate limit login: 5 fail trong 15 phút → lock account 30 phút. Notify user qua email.

RBAC 2 lớp

Lớp 1 — System role (đơn giản, áp dụng global): super_admin (chỉ Trung và team TalentChart), member (mọi người dùng khác).

Lớp 2 — Organization role (chi tiết, áp dụng trong từng org): owner, admin, hr_manager, hr_staff, interviewer, viewer.

```
# backend/app/core/permissions.py

from enum import Enum
from functools import wraps
from fastapi import HTTPException

class SystemRole(str, Enum):
```

```

SUPER_ADMIN = "super_admin"
MEMBER = "member"

class OrgRole(str, Enum):
    OWNER = "owner"          # Chủ tổ chức, full access trong org
    ADMIN = "admin"          # Quản trị tổ chức
    HR_MANAGER = "hr_manager" # Trưởng phòng NS
    HR_STAFF = "hr_staff"     # Nhân viên NS
    INTERVIEWER = "interviewer"
    VIEWER = "viewer"        # Read-only (BGH xem báo cáo)

def require_org_role(*allowed_roles: OrgRole):
    """Decorator yêu cầu user có org_role cụ thể."""
    def decorator(func):
        @wraps(func)
        async def wrapper(*args, current_user, **kwargs):
            if current_user.org_role not in [r.value for r in allowed_roles]:
                raise HTTPException(403, "Insufficient organization role")
            return await func(*args, current_user=current_user, **kwargs)
        return wrapper
    return decorator

```

8.2. Bảo vệ dữ liệu cá nhân (Nghị định 13/2023)

Phân loại dữ liệu trong TalentChart

Loại dữ liệu	Ví dụ	Mức nhạy cảm	Yêu cầu
Định danh cơ bản	Họ tên, email, SĐT	Bình thường	Mã hoá in-transit
Sinh trắc / nhận dạng	Ảnh khuôn mặt	Cao	Mã hoá at-rest
Ngày-giờ-nơi sinh	Cho EPA Engine	Cao	Mã hoá at-rest, opt-in rõ ràng
Tín ngưỡng / mệnh	Mệnh, archetype	Cao	Opt-in, có quyền xoá
Đánh giá hiệu suất	DISC, KPI	Cao	Audit log mọi truy cập

Yêu cầu bắt buộc

- Đồng ý opt-in rõ ràng (granular consent): trước khi thu thập ngày-giờ-nơi sinh, hiển thị popup giải thích mục đích sử dụng và checkbox đồng ý.
- Quyền xoá: user có thể yêu cầu xoá toàn bộ dữ liệu cá nhân — tự động trong 30 ngày, có audit trail.

- Quyền truy xuất: user có thể download toàn bộ dữ liệu của mình (JSON export).
- Mã hoá at-rest: dùng PostgreSQL pgcrypto cho cột sensitive.
- Mã hoá in-transit: TLS 1.3 only, HSTS, Caddy tự động.
- Đăng ký với Cục An toàn Thông tin: trước khi go-live cho khách hàng thứ 2.
- Privacy Policy + Terms of Service: phải có, công khai, dễ hiểu (không legalese).

8.3. OWASP Top 10 mitigation

Vulnerability	Mitigation trong TalentChart
A01: Broken Access Control	Multi-tenant middleware + RLS + permission decorators
A02: Cryptographic Failures	bcrypt cho password, AES-256 cho sensitive fields, TLS 1.3
A03: Injection	SQLAlchemy ORM only, NO raw SQL, Pydantic validation
A04: Insecure Design	Threat model trong Phase 1, security review mọi feature
A05: Security Misconfiguration	Fail-fast config, no defaults for secrets
A06: Vulnerable Components	Dependabot scan weekly, pin major versions
A07: Auth Failures	Rate limit, account lock, MFA Phase 2
A08: Data Integrity Failures	Webhook signing, JWT signature verify
A09: Logging Failures	Structured log + audit log + Sentry
A10: SSRF	Allowlist external URLs, no user-controlled HTTP fetch

Anti-patterns không được phép xuất hiện trong code

DANH SÁCH ĐEN — code review từ chối ngay nếu thấy

- ✗ Raw SQL query với string concatenation (SQL injection)
- ✗ HTML rendering từ user input không escape (XSS)
- ✗ Lookup resource bằng id mà không check tenant ownership (IDOR)
- ✗ Trả về toàn bộ user object qua API (PII leak — phải dùng UserResponse schema)
- ✗ Log chứa password, JWT, hoặc PII nhạy cảm
- ✗ Default secret values trong code
- ✗ CORS allow * cho production
- ✗ Disable CSRF protection cho POST endpoints
- ✗ try/except Exception nuốt error không log

✗ `time.sleep()` trong async code (block event loop)

9. ASYNC, QUEUE, KAFKA — KHI NÀO DÙNG CÁI NÀO

Câu hỏi quan trọng đã được đặt ra

Trung đã hỏi: 'Có nên dùng Kafka không?' Câu trả lời ngắn: Phase 1-2 KHÔNG. Phase 3+ CÂN NHẮC. Chương này giải thích vì sao và đưa ra các lựa chọn thay thế phù hợp với scale Phase 1-2.

9.1. Spectrum của các công cụ async

Có nhiều cấp độ công cụ để xử lý bất đồng bộ. Mỗi cấp độ phù hợp với một quy mô và độ phức tạp khác nhau:

Công cụ	Throughput	Độ phức tạp vận hành	Use case Phase 1-2
FastAPI BackgroundTasks	<10/giây	Rất thấp	Send email, log audit, simple
Redis Pub/Sub	1K/giây	Thấp	Notification real-time, simple events
Redis + RQ (Redis Queue)	1K/giây	Thấp	Background jobs đơn giản
Redis + Celery	10K/giây	Trung bình	Tasks phức tạp (CV parse, PDF, email)
RabbitMQ	30K/giây	Trung bình-Cao	Routing phức tạp giữa services
Kafka	1M+/giây	Cao	Event streaming, data pipeline lớn
NATS	10M+/giây	Trung bình	Microservices messaging

9.2. Quyết định cho TalentChart Phase 1-2

Stack async chốt cho Phase 1-2

Dùng 3 công cụ, không dùng Kafka:

1. FastAPI BackgroundTasks — cho task nhanh và đơn giản (gửi 1 email)
2. Redis + Celery — cho task nặng có thể fail và cần retry (parse CV, sinh PDF, gửi nhiều email)

3. Redis Pub/Sub — cho real-time notification (Phase 2 có thể bổ sung)

Tại sao KHÔNG dùng Kafka cho Phase 1-2?

Kafka là công cụ tuyệt vời cho hệ thống distributed, high-throughput, event-driven. Nhưng dùng Kafka cho TalentChart Phase 1-2 là dùng dao mổ trâu giết kiến. 4 lý do:

Một, throughput không cần. TalentChart Phase 1 với 10 customer pilot ước tính có ~100 event/ngày. Phase 2 với 50 customer trả phí ước tính ~5,000 event/ngày = 0.06 event/giây. Kafka sinh ra cho 1M+ event/giây.

Hai, vận hành phức tạp. Kafka cluster cần ZooKeeper (hoặc KRaft mode), partition planning, broker tuning, replication factor. Junior + 1 mid không thể vận hành Kafka đúng chuẩn cho production. Trung quản lý 12 server đã đủ bận.

Ba, tài nguyên server. Kafka cluster minimal cần 3-4GB RAM cho broker + 1-2GB cho ZooKeeper. Server HPU dành RAM này cho Postgres + Redis có giá trị thực hơn nhiều.

Bốn, tăng độ phức tạp code. Mọi async event với Kafka cần design schema (Avro / Protobuf), version compatibility, dead letter queue, retry policy. Phase 1 chưa xứng đáng đầu tư này.

Khi nào nên cân nhắc Kafka?

Kafka trở nên hữu ích khi xuất hiện ít nhất 1 trong 4 dấu hiệu:

- Throughput vượt 10K event / giây — TalentChart đạt khi >500 customer hoặc B2C app launch (Phase 4).
- Cần event sourcing (rebuild state từ event log) — TalentChart không có nhu cầu này.
- Multiple service consumers cùng đọc 1 event stream — Phase 3 nếu tách microservices.
- Cross-system integration phức tạp với 5+ hệ thống bên ngoài (LMS, SIS, payment gateway, notification service, analytics) — Phase 3-4.

Khi đến thời điểm đó (ước tính tháng 12-18 sau go-live), tôi sẽ viết riêng một tài liệu Kafka migration plan.

9.3. Background jobs với Celery — chi tiết

Celery + Redis là combo chuẩn cho background jobs trong Python. Dùng cho các task nặng hoặc cần retry:

Use case Phase 1-2

Task	Ước tính chạy	Tần suất	Cần Celery?
Gửi email mời test	1-2 giây	Mỗi candidate	Có
Parse CV bằng AI	5-15 giây	Mỗi CV upload	Có
Sinh PDF báo cáo EPA	3-8 giây	Mỗi profile	Có

Task	Ước tính chạy	Tần suất	Cần Celery?
Tính EPA cho 100+ CB hàng loạt	30-60 giây	Khi import	Có
AI matching JD vs CVs	10-30 giây	Mỗi job	Có
Gửi notification real-time	<100ms	Liên tục	Không (Pub/Sub)
Audit log entry	<10ms	Mỗi action	Không (sync OK)

Cấu hình Celery

```
# backend/app/celery_app.py

from celery import Celery
from app.config import settings

celery_app = Celery(
    "talentchart",
    broker=settings.REDIS_URL,
    backend=settings.REDIS_URL,
    include=[
        "app.tasks.email",
        "app.tasks.cv_parse",
        "app.tasks.epa",
        "app.tasks.pdf",
    ]
)

celery_app.conf.update(
    task_serializer="json",
    result_serializer="json",
    accept_content=["json"],
    timezone="Asia/Ho_Chi_Minh",
    enable_utc=True,

    # Retry config
    task_acks_late=True,
    task_reject_on_worker_lost=True,

    # Time limits
    task_soft_time_limit=120, # 2 phút
    task_time_limit=180,      # 3 phút hard limit

    # Worker
    worker_prefetch_multiplier=1, # Quan trọng cho task không đồng đều
    worker_max_tasks_per_child=100, # Tránh memory leak
)
```

Ví dụ task: parse CV với retry

```
# backend/app/tasks/cv_parse.py

from celery import shared_task
from celery.exceptions import SoftTimeLimitExceeded
from anthropic import AsyncAnthropic
from app.config import settings

@shared_task(
    bind=True,
    autoretry_for=(Exception,),
    retry_kwargs={"max_retries": 3, "countdown": 30},
    retry_backoff=True,
)
def parse_cv_task(self, candidate_id: str, cv_url: str):
    """Parse CV PDF/DOCX bằng Claude, extract structured data."""
    try:
        # Download CV from MinIO
        cv_bytes = download_from_minio(cv_url)

        # Call Claude API
        client = AsyncAnthropic(api_key=settings.ANTHROPIC_API_KEY)
        result = await client.messages.create(
            model="claude-sonnet-4-7",
            max_tokens=4000,
            messages=[{
                "role": "user",
                "content": [
                    {"type": "document", "source": {...}},
                    {"type": "text", "text": "Extract..."}
                ]
            }]
        )

        # Save parsed result
        async with async_session_maker() as session:
            candidate = await session.get(Candidate, candidate_id)
            candidate.cv_parsed = result.content[0].text
            await session.commit()

        return {"status": "ok", "candidate_id": candidate_id}

    except SoftTimeLimitExceeded:
        # Cleanup if timeout
```

```
return {"status": "timeout"}
```

9.4. Tóm tắt quyết định async

Khi viết feature mới, hỏi 3 câu để chọn công cụ:

1. Task này chạy <1 giây và không thể fail meaningful?
→ FastAPI BackgroundTasks (đơn giản nhất)
2. Task này chạy >1 giây hoặc cần retry khi fail?
→ Celery task (có monitoring, retry, deadletter)
3. Task này cần real-time push tới client?
→ Redis Pub/Sub + WebSocket (Phase 2)

Nếu không match 3 case trên, hỏi Trung trước khi chọn công cụ khác.

10. DATA IMPORT TỪ DB CŨ

v2.0 — Chiến lược thay đổi từ MIGRATION sang DATA IMPORT

Vì Phase 1 BUILD MỚI hoàn toàn (không migrate code), việc chuyển dữ liệu cũng đơn giản hơn nhiều: export → import. Không cần parallel run, không cần zero-downtime, không cần migration script phức tạp.

10.1. Quy trình tổng thể

Khác với migration code (vốn phải giữ data live), data import là quy trình 1 chiều, có thể làm offline:

Quy trình 5 bước (1 ngày làm việc)

1. Build TalentChart xong, deploy production (hoặc staging)
2. Tạo organization 'HPU' trong TalentChart
3. Export data từ SmartHire DB và Fortune HR DB → JSON files
4. Run import script → load JSON vào TalentChart DB
5. Verify, switch traffic, retire SmartHire & Fortune HR

Toàn bộ quy trình có thể làm trong 1 ngày, có maintenance window 2-4 giờ ban đêm. HPU users báo trước 1 tuần.

10.2. Export từ DB cũ

Script 1 — Export SmartHire (PostgreSQL)

```
# scripts/export_smarthire.py
"""Export SmartHire data to JSON for import to TalentChart."""

import asyncpg
import json
from pathlib import Path

OUTPUT_DIR = Path("./exports/smarthire")
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

async def export_table(conn, table_name: str, query: str):
    rows = await conn.fetch(query)
    data = [dict(row) for row in rows]
```

```

# Convert UUID, datetime, etc. to JSON-safe
for row in data:
    for key, value in row.items():
        if hasattr(value, 'isoformat'):
            row[key] = value.isoformat()
        elif hasattr(value, 'hex'):
            row[key] = str(value)

output_file = OUTPUT_DIR / f"{table_name}.json"
with open(output_file, 'w', encoding='utf-8') as f:
    json.dump(data, f, ensure_ascii=False, indent=2)

print(f" ✓ Exported {len(data)} rows from {table_name} → {output_file}")

async def main():
    conn = await asyncpg.connect(
        "postgresql://smarthire:xxx@old-host:5432/smarthire"
    )

    try:
        # Users
        await export_table(conn, "users", """
            SELECT id, username, email, full_name, hashed_password,
                   role, is_active, avatar_url, created_at, updated_at
            FROM users
        """)

        # Candidates
        await export_table(conn, "candidates", """
            SELECT * FROM candidates
        """)

        # Campaigns
        await export_table(conn, "campaigns", """
            SELECT * FROM campaigns
        """)

        # Test links (chỉ những chưa expired)
        await export_table(conn, "test_links", """
            SELECT * FROM test_links
            WHERE expires_at > NOW()
        """)

        # Interview scores
        await export_table(conn, "interview_scores", """
            SELECT * FROM interview_scores

```

```

        """)

        print("\n✅ Export complete. Files in ./exports/smarthire/")
    finally:
        await conn.close()

if __name__ == "__main__":
    import asyncio
    asyncio.run(main())

```

Script 2 — Export Fortune HR (SQLite)

```

# scripts/export_fortune_hr.py
"""Export Fortune HR data to JSON."""

import sqlite3
import json
from pathlib import Path

OUTPUT_DIR = Path("./exports/fortune_hr")
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

def export_table(conn, table_name: str):
    cursor = conn.execute(f"SELECT * FROM {table_name}")
    columns = [desc[0] for desc in cursor.description]
    rows = [dict(zip(columns, row)) for row in cursor.fetchall()]

    output_file = OUTPUT_DIR / f"{table_name}.json"
    with open(output_file, 'w', encoding='utf-8') as f:
        json.dump(rows, f, ensure_ascii=False, indent=2)

    print(f"  ✓ Exported {len(rows)} rows from {table_name}")

def main():
    conn = sqlite3.connect("/path/to/fortune-hr.db")

    try:
        # Cán bộ + gia đình (Fortune HR coi cả 2 là employees với is_family flag)
        export_table(conn, "employees")
        export_table(conn, "departments")
        # Bỏ qua users (sẽ dùng users từ SmartHire)

        print("\n✅ Export Fortune HR complete.")
    
```

```

    finally:
        conn.close()

if __name__ == "__main__":
    main()

```

10.3. Import vào TalentChart

Script 3 — Import unified

```

# scripts/import_to_talentchart.py
"""
Import từ JSON exports vào TalentChart DB.
Tạo organization 'HPU' và gán mọi data về org này.
"""

import asyncio
import json
import asyncpg
from pathlib import Path
from uuid import uuid4
from app.services.epa.canchi import get_canchi_from_birth
from app.services.epa.archetype import compute_archetype

HPU_ORG_ID = "00000000-0000-0000-0000-000000000001"

async def create_hpu_org(conn):
    """Tạo organization HPU."""
    sql = """
        INSERT INTO organizations (
            id, slug, name, name_short, type, tier,
            contact_email, subdomain, is_active
        )
        VALUES ($1, 'hpu', 'Trường ĐH Quản lý và Công nghệ Hải Phòng',
            'HPU', 'university', 'premier',
            'admin@hpu.edu.vn', 'hpu', TRUE)
        ON CONFLICT (slug) DO NOTHING
    """
    await conn.execute(sql, HPU_ORG_ID)
    print(f"✓ Created/verified organization HPU")

async def import_users(conn):

```

```

"""Import users từ SmartHire export."""
with open("./exports/smarthire/users.json") as f:
    users = json.load(f)

for u in users:
    # Map system role → org role
    org_role = ('owner' if u['role'] == 'super_admin' else
                'admin' if u['role'] == 'hr_manager' else
                'hr_staff' if u['role'] == 'hr_staff' else
                'viewer' if u['role'] == 'bgh' else
                'interviewer')

    await conn.execute("""
        INSERT INTO users (
            id, organization_id, username, email, full_name,
            hashed_password, role, org_role, is_active,
            must_change_password, created_at, updated_at
        )
        VALUES ($1, $2, $3, $4, $5, $6, 'member', $7, $8, TRUE, $9, $10)
        ON CONFLICT DO NOTHING
        """, u['id'], HPU_ORG_ID, u['username'], u['email'], u['full_name'],
            u['hashed_password'], org_role, u['is_active'],
            u['created_at'], u['updated_at'])

print(f"✓ Imported {len(users)} users")

async def import_candidates_from_smarthire(conn):
    """Ứng viên SmartHire → candidates với type='applicant', status mới (7 trạng
    thái)."""
    with open("./exports/smarthire/candidates.json") as f:
        candidates = json.load(f)

    # Map old status → new status
    STATUS_MAP = {
        'new': 'NEW',
        'test_sent': 'TEST_SENT',
        'test_completed': 'TEST_DONE',
        'waiting_interview': 'TEST_DONE', # Gộp
        'interview_round_1': 'INTERVIEW',
        'interview_round_2': 'INTERVIEW',
        'decision': 'DECISION',
        'hired': 'HIRED',
        'rejected': 'REJECTED',
    }

    for c in candidates:

```

```

new_status = STATUS_MAP.get(c['status'], 'NEW')

await conn.execute("""
    INSERT INTO candidates (
        id, organization_id, candidate_type,
        full_name, email, phone, position,
        disc_scores, disc_primary, disc_secondary, disc_profile,
        personality_scores, status, campaign_id,
        created_at, updated_at
    )
    VALUES ($1, $2, 'applicant', $3, $4, $5, $6,
        $7::jsonb, $8, $9, $10, $11::jsonb, $12, $13,
        $14, $15)
""", c['id'], HPU_ORG_ID, c['full_name'], c['email'], c['phone'],
    c['position'], json.dumps(c.get('disc_scores')),
    c['disc_primary'], c['disc_secondary'], c['disc_profile'],
    json.dumps(c.get('personality_scores')),
    new_status, c.get('campaign_id'),
    c['created_at'], c['updated_at'])

print(f"✓ Imported {len(candidates)} candidates from SmartHire")

async def import_employees_from_fortune_hr(conn):
    """Cán bộ Fortune HR → candidates với type='employee' và compute EPA luôn."""
    with open("./exports/fortune_hr/employees.json") as f:
        employees = json.load(f)

    epa_count = 0
    for emp in employees:
        candidate_id = str(uuid4())
        candidate_type = 'employee' if not emp.get('is_family') else 'employee'
        # Note: Fortune HR coi gia đình là entity riêng, ta gộp vào candidates
        # với candidate_type=employee và is_family_member flag

        await conn.execute("""
            INSERT INTO candidates (
                id, organization_id, candidate_type, full_name,
                employee_code, department, is_family_member,
                birth_day, birth_month, birth_year, created_at
            )
            VALUES ($1, $2, 'employee', $3, $4, $5, $6, $7, $8, $9, NOW())
""", candidate_id, HPU_ORG_ID, emp['name'], emp.get('code'),
            emp.get('dept'), bool(emp.get('is_family')),
            emp['day'], emp['month'], emp['year'])

    # Compute EPA profile ngay

```

```

if emp.get('day') and emp.get('month') and emp.get('year'):
    canchi = get_canchi_from_birth(emp['day'], emp['month'], emp['year'])

    # Default DISC neutral (sẽ được update khi nhân viên làm test)
    archetype = compute_archetype(
        disc_profile='S',
        disc_scores={'D': 25, 'I': 25, 'S': 25, 'C': 25},
        canchi=canchi
    )

    await conn.execute("""
        INSERT INTO epa_profiles (
            id, organization_id, candidate_id,
            lunar_year, lunar_month, lunar_day,
            thien_can, dia_chi, con_giap, nap_am, menh,
            archetype_code, archetype_score,
            disc_input, epa_input, algorithm_version
        )
        VALUES ($1, $2, $3, $4, $5, $6, $7, $8, $9, $10, $11,
            $12, $13, $14::jsonb, '{}', '1.0')
    """, str(uuid4()), HPU_ORG_ID, candidate_id,
        canchi['lunar_year'], canchi['lunar_month'], canchi['lunar_day'],
        canchi['thien_can'], canchi['dia_chi'], canchi['con_giap'],
        canchi['nap_am'], canchi['menh'],
        archetype['primary_code'], archetype['primary_score'],
        json.dumps({"D":25,"I":25,"S":25,"C":25}))
    epa_count += 1

print(f"✓ Imported {len(employees)} employees + {epa_count} EPA profiles")

async def main():
    conn = await asyncpg.connect(
        "postgresql://talentchart:xxx@new-host:5432/talentchart"
    )

    try:
        async with conn.transaction():
            await create_hpu_org(conn)
            await import_users(conn)
            await import_candidates_from_smarthire(conn)
            await import_employees_from_fortune_hr(conn)
            # ... import campaigns, test_links, interview_scores

        print("\n✅ Import complete!")
    finally:
        await conn.close()

```

```
if __name__ == "__main__":
    asyncio.run(main())
```

10.4. Verify checklist

- Số lượng users mới = số lượng users export từ SmartHire
- Số lượng candidates type='applicant' = số candidates SmartHire
- Số lượng candidates type='employee' = số employees Fortune HR (đã loại entries lỗi)
- Mọi candidate type='employee' có ngày sinh đều có epa_profile được tính
- Sample test: login bằng tài khoản admin HPU cũ → thấy đầy đủ candidates
- Sample test: xem 1 candidate → thấy DISC scores + interview history còn nguyên
- Sample test: chuyển candidate từ TEST_DONE → INTERVIEW → DECISION → HIRED

10.5. Cutover plan đơn giản

Timeline cutover (chỉ 1 đêm)

20:00 — Maintenance mode trên hire.hpu.edu.vn và tuvi.hpu.edu.vn

20:15 — Final backup 2 DB cũ

20:30 — Run export scripts (SmartHire + Fortune HR → JSON)

21:00 — Run import script → TalentChart DB

21:30 — Verify checklist

22:00 — Update Caddy: hire.hpu.edu.vn → 301 redirect TalentChart

22:15 — Smoke test (login, view, edit)

22:30 — Open TalentChart cho HPU users + email thông báo

Tổng: 2.5 giờ. Hệ cũ giữ read-only 3 tháng phòng cần restore.

11. DEPLOYMENT VỚI CADDY-DOCKER-PROXY

Theo pattern Trung đã có sẵn cho 12 server HPU

Trung đang dùng lucaslorentz/caddy-docker-proxy với label-based routing và external network 'caddy-proxy'. TalentChart áp dụng đúng pattern này — mỗi service tự khai báo domain qua label, không cần sửa Caddyfile khi thêm tenant.

11.1. Pattern caddy-docker-proxy

Caddy-docker-proxy của lucaslorentz là wrapper của Caddy 2 đọc Docker labels để tự động sinh Caddyfile. Lợi ích so với Caddyfile static:

- Mỗi container TỰ KHAI BÁO domain qua label — không cần edit Caddyfile khi deploy service mới.
- Hot reload tự động khi container start/stop — không cần restart Caddy.
- SSL certificate tự động qua DNS challenge (Cloudflare) — kể cả wildcard cho subdomain.
- External network 'caddy-proxy' đã có sẵn — TalentChart chỉ cần join vào.

Caddy-docker-proxy của Trung (đã có)

```
# /root/caddy-project/docker-compose.yml (Trung đã có)

version: '3.9'

services:
  caddy:
    image: lucaslorentz/caddy-docker-proxy:latest
    container_name: caddy
    restart: always

    ports:
      - "80:80"
      - "443:443"

    environment:
      - CADDY_INGRESS_NETWORKS=caddy-proxy

    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
      - ./data:/data
      - ./config:/config

    networks:
      - caddy-proxy
```

```
networks:
  caddy-proxy:
    external: true
```

11.2. TalentChart docker-compose.yml

docker-compose của TalentChart join vào network caddy-proxy và khai báo labels để caddy auto-route:

```
# docker-compose.yml (TalentChart Production)

version: '3.9'

services:
  # — Backend FastAPI —
  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    container_name: talentchart-backend
    restart: unless-stopped

    environment:
      - DATABASE_URL=postgresql+asyncpg://talentchart:${DB_PASSWORD}@db:5432/
talentchart
      - DATABASE_URL_SYNC=postgresql+psycpg2://talentchart:${
{DB_PASSWORD}@db:5432/talentchart
      - REDIS_URL=redis://:${REDIS_PASSWORD}@redis:6379/0
      - JWT_SECRET=${JWT_SECRET}
      - ANTHROPIC_API_KEY=${ANTHROPIC_API_KEY}
      - SMTP_HOST=${SMTP_HOST}
      - SMTP_PORT=${SMTP_PORT}
      - SMTP_USER=${SMTP_USER}
      - SMTP_PASS=${SMTP_PASS}
      - MINIO_ENDPOINT=minio:9000
      - MINIO_ACCESS_KEY=${MINIO_ACCESS_KEY}
      - MINIO_SECRET_KEY=${MINIO_SECRET_KEY}
      - ENV=production
      - LOG_LEVEL=INFO
      - SENTRY_DSN=${SENTRY_DSN}

    depends_on:
      db: { condition: service_healthy }
      redis: { condition: service_healthy }
      minio: { condition: service_healthy }

  networks:
```

```

- talentchart-internal
- caddy-proxy

healthcheck:
  test: ["CMD", "curl", "-f", "http://localhost:8083/api/v1/health"]
  interval: 30s
  timeout: 10s
  retries: 3

# — Frontend Next.js —
frontend:
  build:
    context: ./frontend
    dockerfile: Dockerfile
  container_name: talentchart-frontend
  restart: unless-stopped

environment:
  - NEXT_PUBLIC_API_URL=https://app.talentchart.hpu.edu.vn
  - NODE_ENV=production

depends_on:
  backend: { condition: service_healthy }

networks:
  - talentchart-internal
  - caddy-proxy

# ★ Caddy labels – auto routing
labels:
  # Main app domain (admin)
  caddy_0: app.talentchart.hpu.edu.vn
  caddy_0.reverse_proxy: "{{upstreams 3000}}"

  # Wildcard cho subdomain career pages
  caddy_1: *.talentchart.hpu.edu.vn
  caddy_1.tls: "{env.CLOUDFLARE_API_TOKEN} dns cloudflare"
  caddy_1.reverse_proxy: "{{upstreams 3000}}"

  # API path forwarding (cùng container Next.js làm proxy)
  caddy_1.handle_path_/api/*.0_reverse_proxy: "talentchart-backend:8083"
  caddy_0.handle_path_/api/*.0_reverse_proxy: "talentchart-backend:8083"

# — Celery worker —
worker:
  build:
    context: ./backend

```

```

    dockerfile: Dockerfile
    container_name: talentchart-worker
    restart: unless-stopped
    command: ["celery", "-A", "app.celery_app", "worker", "--loglevel=info", "-Q",
"default"]

    environment:
        # Same as backend
        - DATABASE_URL=postgresql+asyncpg://talentchart:${DB_PASSWORD}@db:5432/
talentchart
        - REDIS_URL=redis://:${REDIS_PASSWORD}@redis:6379/0
        - ANTHROPIC_API_KEY=${ANTHROPIC_API_KEY}
        # ... (other env vars)

    depends_on:
        redis: { condition: service_healthy }
        db: { condition: service_healthy }

    networks:
        - talentchart-internal

# — Celery beat (scheduled tasks) —
beat:
    build:
        context: ./backend
        dockerfile: Dockerfile
    container_name: talentchart-beat
    restart: unless-stopped
    command: ["celery", "-A", "app.celery_app", "beat", "--loglevel=info"]
    environment:
        - REDIS_URL=redis://:${REDIS_PASSWORD}@redis:6379/0
    depends_on:
        redis: { condition: service_healthy }
    networks:
        - talentchart-internal

# — PostgreSQL —
db:
    image: postgres:16-alpine
    container_name: talentchart-db
    restart: unless-stopped
    environment:
        POSTGRES_DB: talentchart
        POSTGRES_USER: talentchart
        POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
        - pgdata:/var/lib/postgresql/data

```

```

- ./scripts/init-db.sql:/docker-entrypoint-initdb.d/01-init.sql:ro
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U talentchart"]
  interval: 5s
  timeout: 5s
  retries: 10
networks:
  - talentchart-internal

# — Redis —
redis:
  image: redis:7-alpine
  container_name: talentchart-redis
  restart: unless-stopped
  command: redis-server --appendonly yes --requirepass ${REDIS_PASSWORD}
  volumes:
    - redisdata:/data
  healthcheck:
    test: ["CMD", "redis-cli", "-a", "${REDIS_PASSWORD}", "ping"]
    interval: 5s
  networks:
    - talentchart-internal

# — MinIO —
minio:
  image: minio/minio:latest
  container_name: talentchart-minio
  restart: unless-stopped
  command: server /data --console-address ":9001"
  environment:
    MINIO_ROOT_USER: ${MINIO_ACCESS_KEY}
    MINIO_ROOT_PASSWORD: ${MINIO_SECRET_KEY}
  volumes:
    - miniodata:/data
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost:9000/minio/health/live"]
    interval: 30s
  networks:
    - talentchart-internal
    - caddy-proxy
  # Optional: expose MinIO console qua subdomain
  labels:
    caddy: storage.talentchart.hpu.edu.vn
    caddy.reverse_proxy: "{{upstreams 9001}}"

# — Backup —
backup:

```

```

    image: postgres:16-alpine
    container_name: talentchart-backup
    restart: unless-stopped
    environment:
      - PGPASSWORD=${DB_PASSWORD}
    volumes:
      - ./backups:/backups
      - ./scripts/backup.sh:/backup.sh:ro
    entrypoint: ["/bin/sh", "-c", "while true; do /backup.sh; sleep 86400; done"]
    depends_on:
      db: { condition: service_healthy }
    networks:
      - talentchart-internal

volumes:
  pgdata:
    name: talentchart-pgdata
  redisdata:
    name: talentchart-redisdata
  miniodata:
    name: talentchart-miniodata

networks:
  talentchart-internal:
    driver: bridge
  caddy-proxy:
    external: true    # Phải có sẵn từ caddy-docker-proxy stack

```

11.3. Backend Dockerfile

```

# backend/Dockerfile

FROM python:3.12-slim AS base

# Install uv (fast Python package manager)
RUN pip install --no-cache-dir uv

WORKDIR /app

# Copy dependency files
COPY pyproject.toml uv.lock* ./

# Install dependencies (production only)
RUN uv sync --frozen --no-dev

```

```
# Copy app code
COPY app/ ./app/
COPY alembic/ ./alembic/
COPY alembic.ini ./

# Run as non-root user
RUN useradd -m -u 1000 talentchart && chown -R talentchart:talentchart /app
USER talentchart

EXPOSE 8083

# Run migrations on startup, then app
CMD ["sh", "-c", "uv run alembic upgrade head && uv run uvicorn app.main:app --host 0.0.0.0 --port 8083 --workers 4"]
```

11.4. Backup script

```
# scripts/backup.sh

#!/bin/sh
set -e

BACKUP_DIR=/backups
DATE=$(date +%Y%m%d-%H%M%S)
RETENTION_DAYS=14

# Postgres dump
pg_dump -h db -U talentchart -d talentchart -F c \
    -f "$BACKUP_DIR/talentchart-$DATE.dump"

# Compress
gzip "$BACKUP_DIR/talentchart-$DATE.dump"

# Remove old
find $BACKUP_DIR -name "talentchart-*.dump.gz" -mtime +$RETENTION_DAYS -delete

# Optional: rsync to remote backup server
# rsync -az "$BACKUP_DIR/" backup-server:/backups/talentchart/

echo "[$(date)] Backup completed: talentchart-$DATE.dump.gz"
```

11.5. Environment variables checklist

.env.production — REQUIRED variables

```
DB_PASSWORD=<random 32+ chars>
REDIS_PASSWORD=<random 32+ chars>
JWT_SECRET=<openssl rand -hex 32>
ANTHROPIC_API_KEY=sk-ant-<your-key>
MINIO_ACCESS_KEY=<random>
MINIO_SECRET_KEY=<random 40+ chars>
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=noreply@hpu.edu.vn
SMTP_PASS=<app-specific password>
CLOUDFLARE_API_TOKEN=<for wildcard SSL>
SENTRY_DSN=<self-hosted sentry URL>
```

App fail-fast khi thiếu bất kỳ var nào.

11.6. Deployment commands

Lần đầu (initial deployment)

```
# Trên server HPU
ssh root@hpu-server

# Clone repo
cd /root/trungth
git clone git@github.com:hpu/talentchart.git
cd talentchart

# Setup .env
cp .env.example .env.production
nano .env.production # Điền các giá trị

# caddy-proxy network đã có sẵn của Trung
docker network ls | grep caddy-proxy # Verify

# Build & start
docker compose --env-file .env.production up -d --build
```

```
# Verify
curl https://app.talentchart.hpu.edu.vn/api/v1/health
# Expected: {"status":"ok","version":"1.0.0"}
```

Update (deploy phiên bản mới)

```
# Trên dev machine
git push origin main

# Trên server HPU
cd /root/trungth/talentchart
git pull origin main

# Rebuild specific services
docker compose --env-file .env.production up -d --build backend frontend worker

# Verify
docker compose logs -f backend | head -50
curl https://app.talentchart.hpu.edu.vn/api/v1/health
```

11.7. Monitoring & alerting

- Uptime Kuma (self-hosted) check /api/v1/health mỗi 1 phút, alert qua Telegram khi down.
- Caddy access log → tail bằng grafana-loki (Phase 2).
- Postgres slow query log: log_min_duration_statement = 1000ms.
- Sentry self-hosted cho error tracking — alert khi error rate tăng 50% so với baseline.
- Daily report qua email: số request, error rate, response time p95.
- Celery monitoring: Flower dashboard tại flower.talentchart.hpu.edu.vn (chỉ admin truy cập).

PHỤ LỤC

A. Quick reference cho dev mới

Cheat sheet — Tham chiếu nhanh

Setup local dev:

```
docker compose -f docker-compose.dev.yml up -d
```

Run migration:

```
docker compose exec backend uv run alembic upgrade head
```

Tạo migration mới:

```
docker compose exec backend uv run alembic revision --autogenerate -m 'description'
```

Run tests:

```
docker compose exec backend uv run pytest -v
```

Format & lint:

```
docker compose exec backend uv run ruff format .
```

```
docker compose exec backend uv run ruff check . --fix
```

```
docker compose exec backend uv run mypy app/
```

Run Celery worker (local):

```
docker compose exec backend uv run celery -A app.celery_app worker --loglevel=info
```

B. Architecture Decision Records (ADR) cần viết

- ADR-001: Build mới hoàn toàn (không migrate code)
- ADR-002: Multi-tenant strategy (shared schema + RLS)
- ADR-003: Modular monolith (không microservices)
- ADR-004: Frontend Next.js 15 App Router
- ADR-005: Hợp nhất bảng employee + candidate (1 bảng, type discriminator)
- ADR-006: Pipeline 7 trạng thái (gọn hơn SmartHire 9)
- ADR-007: Redis + Celery cho async (không Kafka Phase 1-2)

- ADR-008: caddy-docker-proxy với label-based routing

C. Roadmap chương trình huấn luyện team trước Sprint 1

- Tuần 0 — Trung review tài liệu này với team, hỏi đáp 2 buổi (tổng 4h)
- Tuần 0 — Setup local environment cho 3 dev (Docker, Cursor/Claude, Git, repo template)
- Tuần 0 — Trung viết 5 ADR đầu tiên (5 quyết định lớn nhất)
- Tuần 0 — Build skeleton repo: cấu trúc thư mục, Dockerfile, docker-compose, một router 'hello world' để test toàn pipeline
- Tuần 1 — Sprint 1 chính thức bắt đầu (chi tiết ở Tài liệu chiến lược Phase 1-2)